# Geoscientific Machine Learning

Pankaj K Mishra

# Table of Contents

# Preface

Welcome to **Geoscientific Machine Learning**.

If you're reading this, you probably already have some familiarity with geoscience. You're used to thinking in terms of physical processes, models, scales, and uncertainty. When it comes to computation, though, you may have mostly worked through existing tools: running models, adjusting parameters, or modifying scripts written by someone else. That's a common place to be, and it works, until you want to change the model itself or test an idea that doesn't fit into the existing workflow.

You might also be here because you keep hearing about AI and machine learning and you're not sure what to make of it. Is it actually useful for your research, or is it just hype? Will it help you work faster, or will it send you down a rabbit hole? Can it help you do things you thought were out of reach before, or is it mostly a waste of time?

Before you decide, it's worth trying it in a way that respects how geoscience works. That's what this book is about.

By **scientific machine learning**, we mean instead of training a model only on data and hoping it behaves, you guide the learning with what you already know: physical laws, conservation principles, symmetries, and numerical structure. The goal is not just to fit observations, but to produce models that behave sensibly when you change conditions or run them for longer times. By **geoscientific machine learning**, we mean applying scientific machine learning to geoscience problems. That means combining geoscience domain knowledge with learning methods so the models respect physics, scale relationships, and observational constraints.

If you don't yet have a favorite programming language, or you're still deciding what's worth learning, here's the important part: the exact language matters less than having *one*. You need a way to talk to computers clearly, so they can help you turn ideas into experiments and hypotheses into something you can test.

In this book we'll choose **Julia** (Bezanson et al., 2017). If you spend time with it, you'll see why—especially once you start doing more demanding computation. We'll begin slowly and keep things practical, using Julia to express ideas you already know. Machine learning comes later where it connects naturally geoscience practices.

If you're new to programming, expect some friction at first (we'll try to minimise that). That's normal. The payoff is that you gain more control over your models and more confidence in the results they produce.

> **This book is under construction**
>
> Some sections are incomplete. If there's something you'd like to see included, open an issue and let me know what you're working on.

# Part I

# Julia Programming for Geoscience

# 1 Getting started with Julia

You are starting from scratch, and that is perfectly fine. No prior programming experience is assumed.

In this chapter we will install Julia and VS Code, write our first lines of code, and build up to functions, loops, and custom types. By the end you will be comfortable running Julia interactively, saving scripts, and managing packages in reproducible environments. If you have used another language before (Python, R, MATLAB), you will notice how little setup Julia needs to get going.

## 1.1 Install Julia and VS Code

You need two things: **Julia** (the language) and **VS Code** (the editor you will write code in). Install them in this order:

1. Install **VS Code** from code.visualstudio.com.
2. Install **Julia** using juliaup, a small tool that manages Julia versions for you:
   - **Windows (PowerShell)**

     ```
     winget install julia -s msstore
     juliaup add release
     ```

   - **macOS (Terminal)**

     ```
     curl -fsSL https://install.julialang.org | sh
     juliaup add release
     ```

   - **Linux (Terminal)**

     ```
     curl -fsSL https://install.julialang.org | sh
     juliaup add release
     ```

3. Open a new terminal and check that Julia is available:

   ```
   julia --version
   ```

   If it prints something like `julia version 1.11.2`, you are ready.

4. Open VS Code, go to the **Extensions** panel (the square icon on the left sidebar, or press Ctrl/Cmd+Shift+X), and search for **Julia**. Install the one published by **julialang**. This single extension gives you syntax highlighting, code execution, an integrated REPL, a debugger, and a plot viewer. That is everything you need to get started.

> Reading code blocks in this book
>
> Throughout this book you will see two styles of code block. **Julia code** has a thick coloured bar on the left edge:
>
> ```
> println("I am Julia code")
> ```
>
> **Shell commands, configuration files, and other non-Julia text** have a plain border with no coloured bar:
>
> ```
> echo "I am a shell command"
> ```
>
> If a block has the coloured side bar, you can type it into the Julia REPL or a `.jl` file. If it does not, it is meant for your terminal, a config file, or is just illustrative output.

## 1.2 Three ways to run Julia code

Before we start writing code, it helps to know that there are three ways to run Julia. Each is useful in different situations:

### 1.2.1 1. Inside VS Code (recommended for beginners)

This is the easiest way. Create a file ending in `.jl` (e.g., `test.jl`), type your code, and press **Ctrl+Enter** (Windows/Linux) or **Cmd+Enter** (macOS) to run one line or a selected block. The Julia extension opens a built-in REPL panel at the bottom of VS Code and sends your code there. You see the results immediately, and you can go back and edit your file at any time.

This is what we will use throughout this book. It combines the best of both worlds: your code is saved in a file you can come back to, but you can run pieces of it interactively.

### 1.2.2 2. Interactively in the REPL (command line)

Open a terminal and type `julia` to start the **REPL** (Read-Eval-Print Loop). You get a `julia>` prompt where you can type expressions and see results instantly:

```
$ julia
               _
   _       _ _(_)_     |
  (_)     | (_) (_)    |  Version 1.11.2
```

```
   _       _ _(_)_     |
  (_)     | (_) (_)    |
   _ _   _| |_  __ _   |
  | | | | | | |/ _` |  |
  | | |_| | | | (_| |  |
 _/ |\__'_|_|_|\__'_|  |
|__/                   |

julia> 2 + 3
5

julia> sqrt(144)
12.0
```

The REPL is great for quick experiments: testing a formula, checking how a function works, or exploring a dataset. But nothing is saved to a file, so it is not ideal for work you want to keep.

### 1.2.3 3. Running a script from the terminal

If you have a file called `myscript.jl`, you can run the whole thing at once from the terminal:

```
julia myscript.jl
```

Julia reads the file from top to bottom, executes every line, prints any output, and exits. This is how you run production code, batch jobs on a cluster, or automated workflows. You don't see intermediate results, only what the script explicitly prints.

### 1.2.4 Which one should I use?

| Situation | Best option |
|---|---|
| Learning and exploring | VS Code (Ctrl/Cmd+Enter) or REPL |
| Developing code you want to keep | VS Code (code is in a file, but you run it interactively) |
| Quick one-off calculations | REPL |
| Running a finished analysis on a cluster | `julia myscript.jl` |
| Automated or scheduled jobs | `julia myscript.jl` |

You will naturally switch between these as you get more comfortable. For now, open VS Code and follow along.

## 1.3 Hello, Julia!

Let's make sure everything works. In VS Code, create a new file called `test.jl` and type the following:

```julia
println("Hello, Julia!")
x = [1, 2, 3, 4, 5]
println("Sum: ", sum(x))
```

Press **Ctrl+Enter** (Windows/Linux) or **Cmd+Enter** (macOS) to run each line. You should see:

```
Hello, Julia!
Sum: 15
```

Congratulations, you just ran your first Julia program. Everything after this point builds on what you just did.

## 1.4 Your first calculations

Julia works like a calculator right out of the box. You don't need to import anything or set anything up:

```julia
1 + 2, sin(1.0)
```

```
(3, 0.8414709848078965)
```

This computed `1 + 2` and the sine of `1.0` (in radians) and returned both results as a pair. The comma groups them into a **tuple**, just a way of showing multiple answers together.

### 1.4.1 Storing values in variables

In programming, a **variable** is a name you give to a value so you can use it later. Think of it as a sticky note on a number:

```julia
a = 10
b = 3
a + b, a - b, a * b, a / b
```

```
(13, 7, 30, 3.3333333333333335)
```

Here `a` is a label for `10` and `b` is a label for `3`. The next line asks Julia to add, subtract, multiply, and divide them, all at once.

You can name variables almost anything: `temperature`, `depth_km`, `n_samples`. Use names that describe what the value means. Your future self will thank you.

### 1.4.2 Collecting values in a list

In real work you almost never deal with just one number. You have a list of measurements, a column of data, a time series. In Julia, a list of values is called an **Array** (or **Vector** when it is one-dimensional). You create one with square brackets:

```
x = [1, 2, 3, 4, 5]
sum(x), length(x)
```

```
(15, 5)
```

`sum(x)` adds up all the values. `length(x)` counts how many there are. No imports, no setup.

### 1.4.3 What about text?

Not everything is a number. Julia handles text (called **strings**) too. Strings are wrapped in double quotes:

```
name = "Julia"
println("Hello, $name!")
```

The `$` inside a string inserts the value of a variable. This is called **string interpolation**. You can also insert expressions: `"2 + 3 = $(2 + 3)"`.

## 1.5 Reusing code with functions

If you find yourself doing the same calculation over and over, wrap it in a **function**. A function is like a recipe: you give it inputs (ingredients) and it gives you back a result (the dish).

The simplest way to define a function is the one-liner:

```
f(x) = x^2 + 2x + 1
f(3)
```

```
16
```

This says: "define `f` so that it takes `x`, squares it, adds twice `x`, adds `1`, and returns the result." Now `f(3)` gives `16`, `f(10)` gives `121`, and so on.

For anything longer than one line, use the multi-line form:

```
function kinetic_energy(mass, velocity)
    return 0.5 * mass * velocity^2
end
```

```
kinetic_energy(70.0, 3.0)   # a 70 kg person walking at 3 m/s
```

Functions can also return more than one value. Unpack them into separate variables:

```
function minmax(x)
    return minimum(x), maximum(x)
end

lo, hi = minmax([4, 1, 7, 2])
println("min = $lo, max = $hi")
```

> **Why functions matter**
>
> Julia's compiler makes code inside functions run very fast, often as fast as C or Fortran. Code typed at the top level (outside a function) cannot be optimised the same way. So the habit of wrapping your work in functions is not just about tidiness, it is about speed.

## 1.6 Making decisions and repeating things

### 1.6.1 If / else: choosing what to do

Programs often need to choose what to do based on a condition. In Julia, this reads almost like English:

```
x = 5
if x > 0
    println("positive")
elseif x < 0
    println("negative")
else
    println("zero")
end
```

```
positive
```

`println` means "print this line of text." Julia checks the conditions from top to bottom and runs the first one that is true, then skips the rest.

### 1.6.2 Loops: repeating work

A **loop** lets you repeat something many times without copying and pasting. The most common loop says "for each value in this range, do something":

```julia
for i in 1:5
    println(i^2)
end
```

```
1
4
9
16
25
```

`1:5` means the numbers 1, 2, 3, 4, 5. For each one, Julia squares it and prints the result. In geoscience, you might use a loop to process every station in a survey, every layer in a model, or every time step in a simulation.

## 1.7 Organising data with custom types

A geoscience measurement is rarely just a single number. It has coordinates, a value, an uncertainty, maybe a timestamp. Julia lets you bundle related values together using a **struct** (short for structure):

```julia
struct Point
    x::Float64
    y::Float64
end

p = Point(1.0, 2.0)
p.x, p.y
```

```
(1.0, 2.0)
```

This creates a new type called `Point` with two fields. `Float64` means "a decimal number with double precision." Once defined, you create a point by writing `Point(1.0, 2.0)` and access its parts with `p.x` and `p.y`.

> **Why bother with structs?**
>
> As your projects grow, structs keep things organised. Instead of passing around loose variables like `lat`, `lon`, `elevation`, you pass a single `Station` object that holds all of them together. Julia's compiler also uses the type information to make your code faster.

## 1.8 Getting comfortable with the REPL

The **REPL** (Read-Eval-Print Loop) is the interactive prompt you see when you type `julia` in a terminal. It is a great place to try things out quickly.

### 1.8.1 Built-in modes

The REPL has several modes. You switch by pressing a single key at the start of an empty line:

| Key | Mode | What it does |
|---|---|---|
| `]` | **Pkg** | Package manager: add, remove, update packages |
| `;` | **Shell** | Run shell commands without leaving Julia |
| `?` | **Help** | Look up documentation for any function or type |
| Backspace | **Julia** | Return to normal Julia mode |

Try it: press `?` then type `sum` and hit Enter. Julia shows you the documentation for `sum`, with examples.

### 1.8.2 Tab completion and Unicode

Press **Tab** to autocomplete names. This works for functions, variables, file paths, and even Unicode:

- Type `pri` then Tab ⟶ `print`
- Type `\alpha` then Tab ⟶ α
- Type `\sqrt` then Tab ⟶ √

This is how you type mathematical symbols in Julia code. Any LaTeX name works.

### 1.8.3 Handy shortcuts

- **Semicolons suppress output**: `x = rand(1000);` assigns the array without printing all 1000 numbers.
- **`ans` holds the last result**: after typing `2 + 3`, typing `ans * 10` gives `50`.
- **Underscores in numbers**: write `1_000_000` instead of `1000000` for readability. Julia ignores the underscores.

## 1.9 Everyday patterns

These are practical patterns you will use constantly.

### 1.9.1 Broadcasting with the dot `.`

Apply any function or operator to every element of an array by adding a dot:

```
x = [1, 2, 3, 4]
sin.(x)              # sine of each element
x .^ 2               # square each element
x .+ 10              # add 10 to each element
```

This works with your own functions too:

```
f(x) = x^2 + 1
f.([1, 2, 3])        # returns [2, 5, 10]
```

### 1.9.2 Finding documentation and methods

Use `methods()` to see all versions of a function, and `@which` to find out which version gets called:

```
methods(+)               # all methods for +
@which 1.0 + 2.0         # which method handles Float64 + Float64
```

### 1.9.3 Timing your code

Use `@time` for a quick measurement. Run it **twice** because the first call includes compilation time:

```
@time sum(rand(1_000_000))     # first call: includes compilation
@time sum(rand(1_000_000))     # second call: actual speed
```

For serious benchmarking, use the BenchmarkTools package:

```
using BenchmarkTools
@btime sum(rand(1_000_000))
```

## 1.10 Performance habits

These two habits will save you hours of debugging slow code.

### 1.10.1 Put work inside functions

Code at the top level (the global scope) runs slowly because Julia cannot predict the types of global variables. The same code inside a function runs orders of magnitude faster:

```
# Slow — top level
x = rand(1_000_000)
total = 0.0
for val in x
    total += val
end

# Fast — inside a function
function mysum(x)
    total = 0.0
    for val in x
        total += val
    end
    return total
end
mysum(rand(1_000_000))
```

### 1.10.2 Loop in the right order

Julia stores matrices **column by column** in memory (like Fortran), not row by row (like C or Python). When looping over a matrix, always loop over rows in the inner loop:

```
A = rand(1000, 1000)

# Fast — columns in the outer loop, rows in the inner loop
for j in 1:1000       # column
    for i in 1:1000   # row
        A[i, j] += 1
    end
end
```

## 1.11 Customising your Julia setup

### 1.11.1 Startup file

If you load the same packages every session, add them to your startup file. Julia runs this file automatically on launch.

The file lives at `~/.julia/config/startup.jl`. Create the directory and file if they don't exist:

```
mkdir -p ~/.julia/config
echo 'try using Revise catch end' > ~/.julia/config/startup.jl
```

On Windows (PowerShell):

```
New-Item -ItemType Directory -Force -Path "$env:USERPROFILE\.julia\config"
Set-Content "$env:USERPROFILE\.julia\config\startup.jl" 'try using Revise catch end'
```

> **Revise.jl**
>
> Revise watches your source files and automatically picks up changes without restarting Julia. Install it once with `] add Revise`, then load it from your startup file as shown above. It is one of the most useful Julia packages.

### 1.11.2 Point VS Code to Julia manually

In most cases VS Code finds Julia automatically. If it does not, go to **Settings → Julia: Executable Path** and enter the path:

- **Windows**

  `%LOCALAPPDATA%\Programs\Julia-*\bin\julia.exe`

  or

  `%USERPROFILE%\.juliaup\bin\julia.exe`

- **macOS**

  `/Applications/Julia-*.app/Contents/Resources/julia/bin/julia`

  or

  `$HOME/.juliaup/bin/julia`

- **Linux**

  `/usr/bin/julia`

  or

  `$HOME/.juliaup/bin/julia`

| OS | Default location |
|---|---|

## 1.12 The `.julia` directory: where packages live

When you install Julia and start adding packages, Julia creates a hidden directory called `.julia` in your home folder:

| OS | Default location |
|---|---|
| Windows | `C:\Users\YourName\.julia\` |
| macOS | `/Users/YourName/.julia/` |
| Linux | `/home/YourName/.julia/` |

This directory is called the **depot**. It stores everything Julia needs:

| Subfolder | What's inside |
|---|---|
| `packages/` | Source code of every package you install |
| `compiled/` | Precompiled versions of those packages |
| `artifacts/` | Binary libraries and datasets that packages download |
| `registries/` | The General registry, Julia's index of all public packages |
| `logs/` | Build and install logs |
| `environments/` | Project environments and their `Manifest.toml` files |
| `config/` | Your `startup.jl` file |

### 1.12.1 Why it grows large

Every package brings its own dependencies, compiled caches, and sometimes large binary artefacts. A plotting library like CairoMakie can pull in hundreds of megabytes. Over time, `.julia` can easily reach **5–20 GB**.

On a personal laptop this is fine. But on **university HPC clusters and shared Linux servers**, your home directory often has a quota, sometimes as little as 5 or 10 GB. If `.julia` fills it up, you will see "disk quota exceeded" errors and Julia will stop installing packages.

### 1.12.2 Moving `.julia` to a larger disk

Set the `JULIA_DEPOT_PATH` environment variable to point somewhere with more space:

- **Windows (PowerShell, persist permanently)**

```
setx JULIA_DEPOT_PATH "D:\JuliaDepot"
```

- **macOS / Linux** — add to `~/.bashrc` or `~/.zshrc`:

```
export JULIA_DEPOT_PATH="$HOME/julia-depot"
```

- **On a cluster** (use scratch or project space):

```
export JULIA_DEPOT_PATH="/scratch/$USER/julia-depot"
```

Already have a large `.julia` in your home?

Move the existing directory first, then set the variable:

```
mv ~/.julia /scratch/$USER/julia-depot
export JULIA_DEPOT_PATH="/scratch/$USER/julia-depot"
```

On Windows (PowerShell):

```
Move-Item "$env:USERPROFILE\.julia" "D:\JuliaDepot"
setx JULIA_DEPOT_PATH "D:\JuliaDepot"
```

## 1.13 Using Julia behind a proxy

On university and corporate networks, internet traffic often goes through a proxy. Julia needs to know about this to download packages. If you skip this step, `Pkg.add(...)` will hang or time out.

Ask your IT department for the proxy address. Then set it as an environment variable:

**Windows (PowerShell, persist permanently)**

```
setx HTTP_PROXY  "http://proxy.example.com:8080"
setx HTTPS_PROXY "http://proxy.example.com:8080"
```

Close and reopen your terminal after running `setx`.

**macOS / Linux** — add to `~/.bashrc` or `~/.zshrc`:

```
export HTTP_PROXY="http://proxy.example.com:8080"
export HTTPS_PROXY="http://proxy.example.com:8080"
```

Then reload:

```
source ~/.zshrc   # or ~/.bashrc
```

If authentication is required, include credentials in the URL:

```
http://username:password@proxy.example.com:8080
```

### 1.13.1 Verify it works

Open the Julia REPL and try:

```
ENV["HTTP_PROXY"]       # should print your proxy address
using Pkg
Pkg.add("Example")      # should download successfully
```

> No-proxy for local addresses
>
> If internal servers should bypass the proxy:
>
> ```
> export NO_PROXY="localhost,127.0.0.1,.internal.example.com"
> ```
>
> On Windows (PowerShell):
>
> ```
> $env:NO_PROXY = "localhost,127.0.0.1,.internal.example.com"
> ```

## 1.14 Reproducible environments

Imagine you wrote a script that reads geoscientific data, runs an inversion, and produces a beautiful map. It works perfectly today. Six months later your colleague tries to run it and gets errors everywhere. A package was updated, a function was renamed, a dependency now needs a newer Julia version. This is the *"it works on my machine"* problem, and it is one of the biggest obstacles to reproducible science.

Julia's **built-in package manager** solves this at the language level. Every Julia project can carry two small text files that record *exactly* which packages (and which versions) were used. Anyone, anywhere, can recreate the identical environment with a single command.

### 1.14.1 What a Julia project looks like

A typical Julia project is just a folder with a few files:

```
MyProject/
├── Project.toml      # what you depend on
├── Manifest.toml     # exact snapshot of every version
└── src/
    └── MyProject.jl  # your code
```

You may also have a `test/`, `docs/`, or `scripts/` folder, but the two `.toml` files are the heart of reproducibility.

### 1.14.2 `Project.toml`: what your project needs

This file lists the **direct dependencies**, the packages *you* asked for. Here is the `Project.toml` for this very book:

```
[deps]
CSV = "336ed68f-0bac-5ca0-87d4-7b16caf5d00b"
CairoMakie = "13f3f980-e62b-5c42-98c6-ff1f3baf88f0"
DataFrames = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
DelimitedFiles = "8bb1440f-4735-579b-a4ab-409b98df4dab"
Statistics = "10745b16-79ce-11e8-11f9-7d13ad32a3b2"
```

Each entry has a name and a **UUID** (a unique identifier so Julia never confuses two packages with the same name). You rarely type UUIDs by hand; the package manager fills them in when you run `Pkg.add("CSV")`.

You can optionally add a `[compat]` section to pin version ranges:

```
[compat]
CSV = "0.10"
DataFrames = "1.6"
julia = "1.10"
```

This tells Julia: "My code is tested with CSV 0.10.x, DataFrames 1.6.x, and Julia 1.10 or newer." It prevents someone from accidentally installing an incompatible future version.

### 1.14.3 `Manifest.toml`: the exact snapshot

When you install packages, Julia resolves every dependency (and every dependency's dependency) and writes the result into `Manifest.toml`. This file records the **exact version** of every single package in the tree. It can be hundreds of lines long, and that is normal.

Think of it this way:

| File | Analogy | You edit it? |
|---|---|---|
| `Project.toml` | A recipe's ingredient list ("flour, eggs, sugar") | Yes |
| `Manifest.toml` | The supermarket receipt with exact brands and batch numbers | No (generated automatically) |

> Should you commit `Manifest.toml` to Git?
>
> **For applications and papers: yes.** It guarantees bit-for-bit reproducibility. Anyone who checks out your repo and runs `Pkg.instantiate()` gets exactly the same package versions.
> **For libraries (reusable packages): usually no.** You want downstream users to resolve versions against *their* environment, not yours.

### 1.14.4 Creating and using environments

**Start a new project** in any empty folder:

```
using Pkg
Pkg.activate(".")      # use the current folder as the environment
Pkg.add("CSV")         # adds CSV to Project.toml and resolves Manifest.toml
Pkg.add("DataFrames")
```

Or in the REPL's Pkg mode (press `]`):

```
(@v1.11) pkg> activate .
(MyProject) pkg> add CSV DataFrames
```

Julia creates `Project.toml` and `Manifest.toml` for you.

**Reproduce someone else's environment:**

```
using Pkg
Pkg.activate(".")       # point to the folder containing their Project.toml
Pkg.instantiate()       # download the exact versions from Manifest.toml
```

That is it. Two commands and you have an identical setup. No guessing, no version conflicts.

### 1.14.5 The default (global) environment

When you start Julia without activating a project, you are in the **global environment** (shown as `@v1.11` in the REPL prompt). Packages you add here are available everywhere, which is convenient for everyday tools like `Revise` or `BenchmarkTools`. But for actual research work, always create a **project-specific environment** so your results are reproducible.

> How other languages handle this
>
> If you come from **Python**, Julia's `Project.toml` is similar to `requirements.txt` or `pyproject.toml`, and `Manifest.toml` is like a `pip freeze` snapshot or a Poetry/PDM lock file. The difference is that Julia's package manager is built into the language, with no separate tool to install.

If you come from **C++**, reproducible builds are traditionally much harder. You might use CMake with pinned versions, Conan, or vcpkg, but there is no single built-in standard. Julia's approach is closer to Rust's `Cargo.toml` / `Cargo.lock` system.
In all cases the idea is the same: **separate what you want from what you got**, so someone else can recreate the exact same setup later.

### 1.14.6 Key Pkg commands explained

Here is what the most common `Pkg` functions actually do:

`Pkg.activate(".")` **Switch into a project environment.** This tells Julia: "use the `Project.toml` in this folder instead of the global one." The REPL prompt changes from `(@v1.11)` to the project name. Nothing is installed yet; you are just pointing Julia at a folder.

`Pkg.add("CSV")` **Add a package.** Writes the package name and UUID into `Project.toml`, resolves a compatible set of versions, downloads everything needed, and records the result in `Manifest.toml`. If `Project.toml` does not exist yet, Julia creates it for you.

`Pkg.instantiate()` **Recreate an environment from existing files.** If a `Manifest.toml` is present, it downloads the *exact* versions listed there. If only a `Project.toml` exists (no manifest), it resolves fresh versions and creates the manifest. This is the command you run after cloning someone else's repository.

`Pkg.resolve()` **Re-resolve versions without adding or removing anything.** Useful after you edit `Project.toml` by hand (for example, adding a `[compat]` entry). It updates `Manifest.toml` to match the current `Project.toml` without downloading new packages.

`Pkg.update()` **Update all packages** to the newest versions that are still compatible with your `[compat]` constraints. This rewrites `Manifest.toml`. Run this periodically to pick up bug fixes and performance improvements.

`Pkg.status()` **List installed packages** and their versions. In Pkg-mode just type `st` or `status`.

`Pkg.pin("CSV")` **Lock a package at its current version** so that `Pkg.update()` will not change it. Useful when you know a newer release breaks your workflow. Unpin with `Pkg.free("CSV")`.

`Pkg.rm("CSV")` **Remove a package** from the project. Deletes it from `Project.toml` and re-resolves the manifest.

`Pkg.gc()` **Garbage-collect.** Removes cached packages that are no longer used by any environment on your machine. Helpful when your `.julia` folder grows large (see the earlier section on the `.julia` directory).

`Pkg.build()` **Re-run build scripts** for installed packages. Occasionally needed after system-level changes (new compiler, updated shared libraries).

### 1.14.7 Quick reference

| Task | REPL Pkg-mode (]) | Script |
|---|---|---|
| Activate an environment | `activate .` | `Pkg.activate(".")` |
| Add a package | `add CSV` | `Pkg.add("CSV")` |
| Remove a package | `rm CSV` | `Pkg.rm("CSV")` |
| Recreate from lock file | `instantiate` | `Pkg.instantiate()` |
| Re-resolve after hand edits | `resolve` | `Pkg.resolve()` |

| Task | REPL Pkg-mode (]) | Script |
|---|---|---|
| See what is installed | `status` | `Pkg.status()` |
| Update all packages | `update` | `Pkg.update()` |
| Pin a version | `pin CSV@0.10.14` | `Pkg.pin("CSV", v"0.10.14")` |
| Unpin | `free CSV` | `Pkg.free("CSV")` |
| Clean unused caches | `gc` | `Pkg.gc()` |
| Re-run build scripts | `build` | `Pkg.build()` |

Learn more

The official **Pkg documentation** covers everything in detail, including custom registries, private packages, artifacts, and more:
https://pkgdocs.julialang.org/

With these two files and a handful of commands, your work stays reproducible, whether you revisit it next year or share it with a colleague on the other side of the world.

# 2 Files & Data Manipulation

At this point you know how to install Julia, run code in VS Code or the REPL, write functions, and create reproducible environments with `Project.toml`.

In this chapter we will put those skills to work on something you will do constantly in geoscience: **reading data from files, reshaping it, and writing results back out**. We start with plain text files, move on to delimited numeric data, then to CSV tables with the `DataFrames` ecosystem, and finish with a tour of specialised geoscientific formats (NetCDF, HDF5, SEG-Y, Shapefiles). We create small example files right here in the code so you can run every block and see the results yourself.

If some of the syntax looks unfamiliar at first, especially the `do ... end` pattern or the ⇒ arrow, don't worry. These are common Julia patterns that will become second nature with practice. The important thing is to understand *what* each block does, not to memorise the syntax right now.

## 2.1 Writing and reading text files

The simplest kind of file is plain text. Let's create one:

```
# Create a small text file
open("stations.txt", "w") do f
    write(f, "Station  Latitude  Longitude  Elevation_m\n")
    write(f, "AAL      64.35     25.75      120\n")
    write(f, "BBR      61.50     23.80      95\n")
    write(f, "CCK      60.17     24.94      15\n")
end
```

```
33
```

What happened here? `open("stations.txt", "w")` creates (or overwrites) a file named `stations.txt`. The `"w"` means "write mode." The `do f ... end` block is Julia's way of saying "while the file is open, call it `f` and do these things, then close it automatically." Each `write` call puts a line of text into the file.

Now let's read it back:

```
content = read("stations.txt", String)
println(content)
```

```
Station  Latitude  Longitude  Elevation_m
AAL      64.35     25.75      120
BBR      61.50     23.80      95
CCK      60.17     24.94      15
```

`read("stations.txt", String)` loads the entire file into a single string. For a small file like this, that is perfectly fine.

### 2.1.1 Reading line by line

Sometimes you want to process a file one line at a time, for instance to skip a header or to parse each row:

```
open("stations.txt", "r") do f
    for line in eachline(f)
        println("→ ", line)
    end
end
```

```
→ Station  Latitude  Longitude  Elevation_m
→ AAL      64.35     25.75      120
→ BBR      61.50     23.80      95
→ CCK      60.17     24.94      15
```

The `"r"` means "read mode." `eachline(f)` gives you one line at a time, which is memory-efficient even for very large files.

### 2.1.2 Splitting lines into columns

In geoscience data files, columns are usually separated by spaces, tabs, or commas. You can split a line into pieces with `split()`:

```
open("stations.txt", "r") do f
    header = readline(f)              # read and skip the header
    for line in eachline(f)
        parts = split(line)           # splits on whitespace by default
        name  = parts[1]
        lat   = parse(Float64, parts[2])
        lon   = parse(Float64, parts[3])
        elev  = parse(Float64, parts[4])
        println("$name is at ($lat, $lon), elevation $elev m")
    end
end
```

```
AAL is at (64.35, 25.75), elevation 120.0 m
BBR is at (61.5, 23.8), elevation 95.0 m
CCK is at (60.17, 24.94), elevation 15.0 m
```

`split(line)` breaks the string into a list of substrings. `parse(Float64, ...)` converts a text string like `"64.35"` into an actual number. This pattern (read a line, split it, parse the pieces) is the bread and butter of working with geoscience text files.

## 2.2 Delimited data with `DelimitedFiles`

For files with a regular grid of numbers (like gravity measurements or temperature readings), Julia has a built-in module called `DelimitedFiles` that reads the whole file into a matrix in one step.

Let's create a small data file and read it:

```julia
# Create a space-delimited data file (no header, just numbers)
open("gravity.txt", "w") do f
    write(f, "64.35  25.75  -12.3\n")
    write(f, "64.40  25.80  -11.8\n")
    write(f, "64.45  25.85  -13.1\n")
    write(f, "64.50  25.90  -12.7\n")
    write(f, "64.55  25.95  -14.0\n")
end
```

```
20
```

```julia
using DelimitedFiles

data = readdlm("gravity.txt")       # reads into a matrix
println("Size: ", size(data))        # 5 rows, 3 columns
```

```
Size: (5, 3)
```

Now you can pull out individual columns:

```julia
lat = data[:, 1]         # all rows, first column
lon = data[:, 2]         # all rows, second column
gz  = data[:, 3]         # all rows, third column

println("Latitudes: ", lat)
println("Gravity anomalies: ", gz)
```

```
Latitudes: [64.35, 64.4, 64.45, 64.5, 64.55]
Gravity anomalies: [-12.3, -11.8, -13.1, -12.7, -14.0]
```

The `[:, 1]` notation means "all rows, column 1." This is the same notation used in MATLAB and NumPy.

You can also write a matrix back out:

```julia
# Add 1.0 to all gravity values and save
gz_corrected = gz .+ 1.0
output = hcat(lat, lon, gz_corrected)    # glue columns side by side
writedlm("gravity_corrected.txt", output, '\t')   # tab-separated

# Verify
println(read("gravity_corrected.txt", String))
```

```
64.35   25.75   -11.3
64.4    25.8    -10.8
64.45   25.85   -12.1
64.5    25.9    -11.7
64.55   25.95   -13.0
```

`hcat` stands for "horizontal concatenate". It sticks columns together into a matrix. `writedlm` writes the matrix to a file, using the tab character `'\t'` as the separator.

## 2.3 CSV files and DataFrames

CSV (Comma-Separated Values) is probably the most common data format in science. Julia has excellent CSV support through the **CSV.jl** and **DataFrames.jl** packages.

> **Installing packages**
>
> If you haven't installed these packages yet, press `]` in the REPL to enter Pkg mode and type:
>
> `add CSV DataFrames`
>
> Or from normal Julia code: `using Pkg; Pkg.add(["CSV", "DataFrames"])`. You only need to do this once.

### 2.3.1 Creating a DataFrame

A **DataFrame** is a table: rows and columns, like a spreadsheet. Each column has a name and all values in a column have the same type. If you have used pandas in Python, R's `data.frame`, or even Excel, the concept is the same.

```
using DataFrames

# Create a table of borehole samples
samples = DataFrame(
    borehole  = ["BH-01", "BH-01", "BH-01", "BH-02", "BH-02"],
    depth_m   = [10.0, 20.0, 30.0, 15.0, 25.0],
    rock_type = ["granite", "granite", "gneiss", "granite", "schist"],
    density   = [2.65, 2.68, 2.75, 2.63, 2.71]
)
```

| | borehole | depth_m | rock_type | density |
|---|---|---|---|---|
| | String | Float64 | String | Float64 |
| 1 | BH-01 | 10.0 | granite | 2.65 |
| 2 | BH-01 | 20.0 | granite | 2.68 |
| 3 | BH-01 | 30.0 | gneiss | 2.75 |
| 4 | BH-02 | 15.0 | granite | 2.63 |
| 5 | BH-02 | 25.0 | schist | 2.71 |

Julia displays the table neatly. Each column is a named array, and you can work with them individually or together.

### 2.3.2 Writing and reading CSV

Let's save this table to a CSV file and read it back:

```
using CSV

# Write to CSV
CSV.write("samples.csv", samples)

# Read it back
samples_loaded = CSV.read("samples.csv", DataFrame)
```

| | borehole | depth_m | rock_type | density |
|---|---|---|---|---|
| | String7 | Float64 | String7 | Float64 |
| 1 | BH-01 | 10.0 | granite | 2.65 |
| 2 | BH-01 | 20.0 | granite | 2.68 |
| 3 | BH-01 | 30.0 | gneiss | 2.75 |
| 4 | BH-02 | 15.0 | granite | 2.63 |
| 5 | BH-02 | 25.0 | schist | 2.71 |

That's it. One line to write, one line to read. CSV.jl automatically detects column types, handles headers, and deals with missing values.

### 2.3.3 Selecting columns

Grab one or more columns by name:

```
# One column - returns a vector
samples.depth_m
```

```
5-element Vector{Float64}:
 10.0
 20.0
 30.0
 15.0
 25.0
```

```
# Multiple columns - returns a new DataFrame
select(samples, :borehole, :density)
```

| | borehole | density |
|---|---|---|
| | String | Float64 |
| 1 | BH-01 | 2.65 |
| 2 | BH-01 | 2.68 |
| 3 | BH-01 | 2.75 |
| 4 | BH-02 | 2.63 |
| 5 | BH-02 | 2.71 |

The `:` before a column name makes it a **Symbol**, Julia's way of referring to names. You will see this pattern everywhere in DataFrames.

### 2.3.4 Filtering rows

Keep only rows that match a condition:

```
# Samples deeper than 20 m
filter(row -> row.depth_m > 20, samples)
```

| | borehole | depth_m | rock_type | density |
|---|---|---|---|---|
| | String | Float64 | String | Float64 |
| 1 | BH-01 | 30.0 | gneiss | 2.75 |
| 2 | BH-02 | 25.0 | schist | 2.71 |

The `row -> row.depth_m > 20` part is an **anonymous function**, a small throwaway function that takes one argument (`row`) and returns `true` or `false`. Don't worry about the syntax too much; think of it as saying "keep rows where depth is greater than 20."

```
# Only granite samples
filter(row -> row.rock_type == "granite", samples)
```

| | borehole | depth_m | rock_type | density |
|---|---|---|---|---|
| | String | Float64 | String | Float64 |
| 1 | BH-01 | 10.0 | granite | 2.65 |
| 2 | BH-01 | 20.0 | granite | 2.68 |
| 3 | BH-02 | 15.0 | granite | 2.63 |

### 2.3.5 Adding and transforming columns

```
# Add a new column
samples.weight_kg = samples.density .* 1000.0   # density × 1000
samples
```

| | borehole | depth_m | rock_type | density | weight_kg |
|---|---|---|---|---|---|
| | String | Float64 | String | Float64 | Float64 |
| 1 | BH-01 | 10.0 | granite | 2.65 | 2650.0 |
| 2 | BH-01 | 20.0 | granite | 2.68 | 2680.0 |
| 3 | BH-01 | 30.0 | gneiss | 2.75 | 2750.0 |
| 4 | BH-02 | 15.0 | granite | 2.63 | 2630.0 |
| 5 | BH-02 | 25.0 | schist | 2.71 | 2710.0 |

The `.*` (dot-star) applies the multiplication to every row. You saw this broadcasting pattern in Chapter 1.

### 2.3.6 Grouping and summarising

This is where DataFrames really shine. Group your data by a category and compute summaries:

```
using Statistics

# Average density per borehole
gdf = groupby(samples, :borehole)
combine(gdf, :density ⇒ mean ⇒ :avg_density)
```

| | borehole | avg_density |
|---|---|---|
| | String | Float64 |
| 1 | BH-01 | 2.69333 |
| 2 | BH-02 | 2.67 |

Read this as: "group the table by `:borehole`, then for the `:density` column compute the `mean` and call the result `:avg_density`." The ⇒ arrows connect input → function → output name.

```
# Multiple summaries at once
combine(
    groupby(samples, :rock_type),
    :density ⇒ mean ⇒ :avg_density,
    :density ⇒ minimum ⇒ :min_density,
    :depth_m ⇒ maximum ⇒ :deepest
)
```

| | rock_type | avg_density | min_density | deepest |
|---|---|---|---|---|
| | String | Float64 | Float64 | Float64 |
| 1 | granite | 2.65333 | 2.63 | 20.0 |
| 2 | gneiss | 2.75 | 2.75 | 30.0 |
| 3 | schist | 2.71 | 2.71 | 25.0 |

### 2.3.7 Joining tables

In real projects your data is often spread across multiple files. Joining brings them together:

```
# A second table with borehole coordinates
locations = DataFrame(
    borehole  = ["BH-01", "BH-02", "BH-03"],
    latitude  = [64.35, 64.40, 64.45],
    longitude = [25.75, 25.80, 25.85]
)

# Inner join - only boreholes that appear in both tables
innerjoin(samples, locations, on = :borehole)
```

| | borehole | depth_m | rock_type | density | weight_kg | latitude | longitude |
|---|---|---|---|---|---|---|---|
| | String | Float64 | String | Float64 | Float64 | Float64 | Float64 |
| 1 | BH-01 | 10.0 | granite | 2.65 | 2650.0 | 64.35 | 25.75 |
| 2 | BH-01 | 20.0 | granite | 2.68 | 2680.0 | 64.35 | 25.75 |
| 3 | BH-01 | 30.0 | gneiss | 2.75 | 2750.0 | 64.35 | 25.75 |
| 4 | BH-02 | 15.0 | granite | 2.63 | 2630.0 | 64.4 | 25.8 |
| 5 | BH-02 | 25.0 | schist | 2.71 | 2710.0 | 64.4 | 25.8 |

`innerjoin` keeps only rows where the `:borehole` value exists in both tables. BH-03 has no samples, so it is dropped. If you want to keep all locations even when there are no matching samples, use `leftjoin` or `outerjoin` instead.

### 2.3.8 Handling missing data

Real data has gaps. Julia represents missing values with a special value called `missing`:

```
# Create data with a gap
obs = DataFrame(
    station = ["A", "B", "C", "D"],
    temp_C  = [12.3, missing, 14.1, 13.7]
)
obs
```

| | station | temp_C |
|---|---|---|
| | String | Float64? |
| 1 | A | 12.3 |
| 2 | B | *missing* |
| 3 | C | 14.1 |
| 4 | D | 13.7 |

```
# Drop rows with missing values
dropmissing(obs, :temp_C)
```

| | station | temp_C |
|---|---|---|
| | String | Float64 |
| 1 | A | 12.3 |
| 2 | C | 14.1 |
| 3 | D | 13.7 |

```
# Compute mean, skipping missing values
using Statistics
mean(skipmissing(obs.temp_C))
```

```
13.366666666666665
```

`skipmissing` is a wrapper that tells functions to ignore the gaps. Without it, `mean` would return `missing` because any operation involving `missing` propagates the unknown.

## 2.4 Working with file paths

When your project has many data files, you need to build file paths correctly. Julia's `joinpath` function handles this across operating systems (so you don't have to worry about `/` vs `\`):

```
# Build a path
path = joinpath("data", "field_campaign", "gravity.txt")
println(path)
```

```
data/field_campaign/gravity.txt
```

Other useful path functions:

```
println("Current directory: ", pwd())
println("Files here: ", readdir("."))
```

```
# Check if a file exists before reading
if isfile("samples.csv")
    println("samples.csv exists!")
else
    println("File not found")
end
```

```
samples.csv exists!
```

> **Organise your data early**
>
> Create a `data/` folder in your project from the start. Put raw data in `data/raw/`, processed data in `data/processed/`. This habit will save you from chaos later, especially when you have 50 stations and 3 field campaigns.

## 2.5 Searching, copying, and renaming files

In practice, geoscientific data rarely arrives in one tidy folder. You might receive a USB drive with hundreds of files scattered across nested directories, including raw measurements from field instruments, GPS logs, photos, metadata files, all mixed together. A common first task is: **find all files of a certain type, rename them consistently, and copy them into a single clean folder.**

Julia has everything you need for this built into the standard library, no extra packages required.

### 2.5.1 Listing files in a directory

```
# readdir returns the names of everything in a folder
for name in readdir(".")
    println(name)
end
```

```
01-getting-started-julia.html
01-getting-started-julia.qmd
01-getting-started-julia_files
02-files-data.html
02-files-data.qmd
```

```
02-files-data_files
03-plotting-maps.qmd
03-plotting-maps_files
04-neural-networks.qmd
04-neural-networks_files
05-inverse-modeling.qmd
06-applications.qmd
gravity.txt
gravity_corrected.txt
samples.csv
sauna_log.txt
stations.txt
```

By default `readdir` gives you the names without the full path. Pass `join = true` to get complete paths:

```
for path in readdir("data/raw", join = true)
    println(path)
end
```

### 2.5.2 Searching recursively with `walkdir`

`walkdir` is the key function for searching through directories and all their subdirectories. It yields a tuple `(root, dirs, files)` at each level:

```
# Find every .csv file anywhere inside data/
for (root, dirs, files) in walkdir("data")
    for f in files
        if endswith(f, ".csv")
            full_path = joinpath(root, f)
            println(full_path)
        end
    end
end
```

You can wrap this into a reusable function:

```
"""
    find_files(dir, ext)

Recursively find all files with extension `ext` in `dir` and its subdirectories.
Returns a vector of full paths.
"""
function find_files(dir, ext)
    results = String[]
    for (root, dirs, files) in walkdir(dir)
```

```
        for f in files
            if endswith(f, ext)
                push!(results, joinpath(root, f))
            end
        end
    end
    return results
end
```

```
Main.Notebook.find_files
```

```
# Example: find all .dat files under a field campaign folder
dat_files = find_files("data/raw", ".dat")
println("Found $(length(dat_files)) .dat files")
```

### 2.5.3 Extracting parts of a file path

Julia has several functions for splitting a path into its components:

```
example_path = joinpath("surveys", "2024", "gravity", "station_042.csv")

println("Directory:  ", dirname(example_path))
println("Filename:   ", basename(example_path))
println("Name only:  ", splitext(basename(example_path))[1])
println("Extension:  ", splitext(basename(example_path))[2])
println("Split path: ", splitpath(example_path))
```

```
Directory:  surveys/2024/gravity
Filename:   station_042.csv
Name only:  station_042
Extension:  .csv
Split path: ["surveys", "2024", "gravity", "station_042.csv"]
```

### 2.5.4 Copying and renaming files

Suppose you have collected EM data from multiple campaigns and the instrument saved files with unhelpful names like `meas_001.dat`. You want to copy them into a single `processed/` folder, renaming each file to include the station name and date.

Here is a complete worked example. We first create a fake directory tree so you can run everything:

```
# --- Set up a fake directory tree ---
for station in ["EM01", "EM02", "EM03"]
    dir = joinpath("fake_survey", station)
    mkpath(dir)    # like mkdir -p: creates all intermediate directories
    for i in 1:3
        filepath = joinpath(dir, "meas_$(lpad(i, 3, '0')).dat")
        write(filepath, "fake data for $station measurement $i\n")
    end
end
println("Created fake survey tree:")
for (root, dirs, files) in walkdir("fake_survey")
    for f in files
        println("  ", joinpath(root, f))
    end
end
```

```
Created fake survey tree:
  fake_survey/EM01/meas_001.dat
  fake_survey/EM01/meas_002.dat
  fake_survey/EM01/meas_003.dat
  fake_survey/EM02/meas_001.dat
  fake_survey/EM02/meas_002.dat
  fake_survey/EM02/meas_003.dat
  fake_survey/EM03/meas_001.dat
  fake_survey/EM03/meas_002.dat
  fake_survey/EM03/meas_003.dat
```

Now let's search for all `.dat` files, rename them to include the station name, and copy them into a clean output folder:

```
# --- Find, rename, and copy ---
output_dir = "collected_data"
mkpath(output_dir)

for (root, dirs, files) in walkdir("fake_survey")
    for f in files
        if endswith(f, ".dat")
            # The station name is the immediate parent folder
            station = basename(root)

            # Build a new name: EM01_meas_001.dat
            new_name = station * "_" * f

            src = joinpath(root, f)
            dst = joinpath(output_dir, new_name)
```

```
            cp(src, dst, force = true)   # copy; force=true overwrites if exists
        end
    end
end

# Check the result
println("Files in $(output_dir):")
for f in sort(readdir(output_dir))
    println("  ", f)
end
```

```
Files in collected_data:
  EM01_meas_001.dat
  EM01_meas_002.dat
  EM01_meas_003.dat
  EM02_meas_001.dat
  EM02_meas_002.dat
  EM02_meas_003.dat
  EM03_meas_001.dat
  EM03_meas_002.dat
  EM03_meas_003.dat
```

### 2.5.5 Key functions at a glance

| Function | What it does |
|---|---|
| `readdir(dir)` | List contents of a directory |
| `readdir(dir, join=true)` | Same, but returns full paths |
| `walkdir(dir)` | Recursively walk through all subdirectories |
| `mkpath(dir)` | Create a directory (and parents), like `mkdir -p` |
| `cp(src, dst)` | Copy a file |
| `mv(src, dst)` | Move or rename a file |
| `rm(path)` | Delete a file |
| `rm(path, recursive=true)` | Delete a directory and everything inside it |
| `isfile(path)` | Check if a path points to an existing file |
| `isdir(path)` | Check if a path points to an existing directory |
| `joinpath(parts...)` | Build a path from pieces (cross-platform) |
| `basename(path)` | Filename from a path |
| `dirname(path)` | Directory from a path |
| `splitext(name)` | Split into `(name, ".ext")` |
| `splitpath(path)` | Split into a vector of path components |

## 2.6 Geoscientific file formats

Geoscience uses several specialised binary formats to store large, multidimensional datasets efficiently. Julia has packages for all the common ones. Here is a brief overview. Detailed usage will be added in future updates of this chapter.

### 2.6.1 NetCDF

NetCDF (Network Common Data Form) is the standard format for climate, ocean, and atmospheric data. Use the **NCDatasets.jl** package:

```julia
using NCDatasets

# Read a NetCDF file
ds = NCDataset("temperature.nc")
temp = ds["temperature"][:, :, :]    # read the full 3D array
lat  = ds["latitude"][:]
lon  = ds["longitude"][:]
close(ds)

# Or use the do-block pattern (auto-closes)
NCDataset("temperature.nc") do ds
    temp = ds["temperature"][:, :, :]
    println("Shape: ", size(temp))
end
```

```julia
# Write a NetCDF file
NCDataset("output.nc", "c") do ds
    defDim(ds, "x", 10)
    defDim(ds, "y", 20)
    v = defVar(ds, "elevation", Float64, ("x", "y"))
    v[:, :] = rand(10, 20)
end
```

### 2.6.2 HDF5

HDF5 (Hierarchical Data Format) is widely used in geophysics, remote sensing, and seismology. Use **HDF5.jl**:

```julia
using HDF5

# Write
h5open("model.h5", "w") do f
    f["resistivity"] = rand(100, 100, 50)
```

```
    f["depth"] = collect(0.0:0.5:24.5)
end

# Read
h5open("model.h5", "r") do f
    rho   = read(f, "resistivity")
    depth = read(f, "depth")
    println("Model size: ", size(rho))
end
```

### 2.6.3 SEG-Y

SEG-Y is the seismic industry standard. Use **SegyIO.jl**:

```
using SegyIO

# Read a SEG-Y file
block = segy_read("seismic_line.segy")
traces = block.data           # matrix of traces
headers = block.traceheaders   # trace header information
```

### 2.6.4 Shapefiles and GeoJSON

For vector geospatial data (points, polygons, map boundaries), use **Shapefile.jl** or **GeoJSON.jl**:

```
using Shapefile

table = Shapefile.Table("geology_map.shp")
# Access as a DataFrame
using DataFrames
df = DataFrame(table)
```

⚠ These examples are not executed

The geoscientific format examples above use `eval: false` style. They show the code patterns but don't run here because they require actual data files. When you have your own NetCDF, HDF5, or SEG-Y files, copy the pattern and replace the file name.

! Want more formats or deeper coverage?

This section will grow as the book develops. If there is a specific geoscience format or workflow you need (GRIB, GeoTIFF, ASDF, miniSEED, EDI, ModEM, or anything else) please open an issue on

GitHub and describe your use case. Your request will help us prioritise what to add next.

## 2.7 Clean up

Let's remove the temporary files we created in this chapter:

```
for f in ["stations.txt", "gravity.txt", "gravity_corrected.txt", "samples.csv"]
    isfile(f) && rm(f)
end
println("Cleaned up.")
```

```
Cleaned up.
```

# 3 Plotting & Data Visualisation

At this point you know how to write Julia code, work with functions and data structures, and read data from files into arrays and DataFrames.

In this chapter we will learn how to **visualise** that data. We start from the very basics, a single line on a pair of axes, and build up to the kinds of plots geoscientists use every day: scatter plots with error bars, borehole logs, contour maps, heatmaps, multi-panel figures, and more. By the end you will be able to turn any dataset into a publication-ready figure and save it as PNG, PDF, or SVG.

## 3.1 The Makie ecosystem

Julia has several plotting libraries. In this book we use **Makie**, a modern, high-performance plotting ecosystem designed for scientific visualisation. Makie is split into **backends**. You write the same plotting code, but choose a backend depending on where you want the output:

| Backend | Package | Best for |
|---|---|---|
| **CairoMakie** | `CairoMakie.jl` | Static figures (PNG, PDF, SVG) for papers, reports, this book |
| **GLMakie** | `GLMakie.jl` | Interactive, GPU-accelerated windows for rotating 3-D models, exploring large datasets on your screen |
| **WGLMakie** | `WGLMakie.jl` | Interactive plots in a web browser or Jupyter/Pluto notebook |

> **!** CairoMakie vs GLMakie: which one do I use?
>
> Think of Makie as one plotting language with different "printers":
>
> - **CairoMakie** renders to a file (PNG, PDF, SVG). There is no window. You call `save("fig.pdf", fig)` and get a crisp vector graphic ready for a journal. It works everywhere, including headless servers and HPC clusters with no display. This is what we use throughout this book.
> - **GLMakie** opens an **interactive window** on your screen, powered by your GPU. You can pan, zoom, and rotate 3-D scenes with the mouse, perfect for exploring a velocity model, a point cloud, or a seismicity catalogue before you decide which view to export. You need a display and a working OpenGL driver.
> - **WGLMakie** does the same interactive thing, but inside a **web browser**. It is the best choice for Jupyter or Pluto notebooks.

The key point: **your plotting code does not change**. You just swap `using CairoMakie` for `using GLMakie` (or `using WGLMakie`). So learn the API once, and pick the backend that fits the task.

For this book we use **CairoMakie** because it produces crisp vector graphics and does not need a GPU. When you start exploring 3-D geological models or large point clouds interactively, switch to **GLMakie**. The plotting code stays the same, only the `using` line changes.

What about Plots.jl and Plotly?

**Plots.jl** is another popular Julia plotting package with a different API. It supports multiple backends (GR, Plotly, PGFPlotsX, etc.) through a single interface. If you prefer Plotly-style interactive HTML plots, you can use `Plots.jl` with the `plotlyjs()` backend, or use **PlotlyJS.jl** directly.
We chose Makie for this book because its composability (building complex figures from simple pieces) and performance with large datasets make it a natural fit for geoscientific work. But there is no wrong choice — explore both and use what feels right for your workflow.

GMT.jl: the Generic Mapping Tools for Julia

If you work with geographic or bathymetric data, you should know about **GMT.jl**, a Julia wrapper around the legendary Generic Mapping Tools (GMT). GMT has been the gold standard for map-making in the earth sciences for over 30 years. The Julia wrapper gives you the full power of GMT (coastlines, topographic relief, geodetic projections, gravity grids, focal mechanisms) from within Julia.
GMT.jl is not covered in this chapter because it has its own extensive documentation and a different API from Makie. But if your work involves publication-quality maps with coastlines, plate boundaries, or bathymetry, it is absolutely worth exploring:
https://www.generic-mapping-tools.org/GMT.jl/stable/

## 3.2 Installing CairoMakie

Before we can plot anything, we need to add CairoMakie to our project environment. Press `]` in the REPL to enter Pkg mode:

```
pkg> add CairoMakie
```

Or from normal Julia code:

```
using Pkg; Pkg.add("CairoMakie")
```

You only need to do this once per environment. CairoMakie is already in this book's `Project.toml`, so if you cloned the book repository and ran `Pkg.instantiate()`, you are all set.

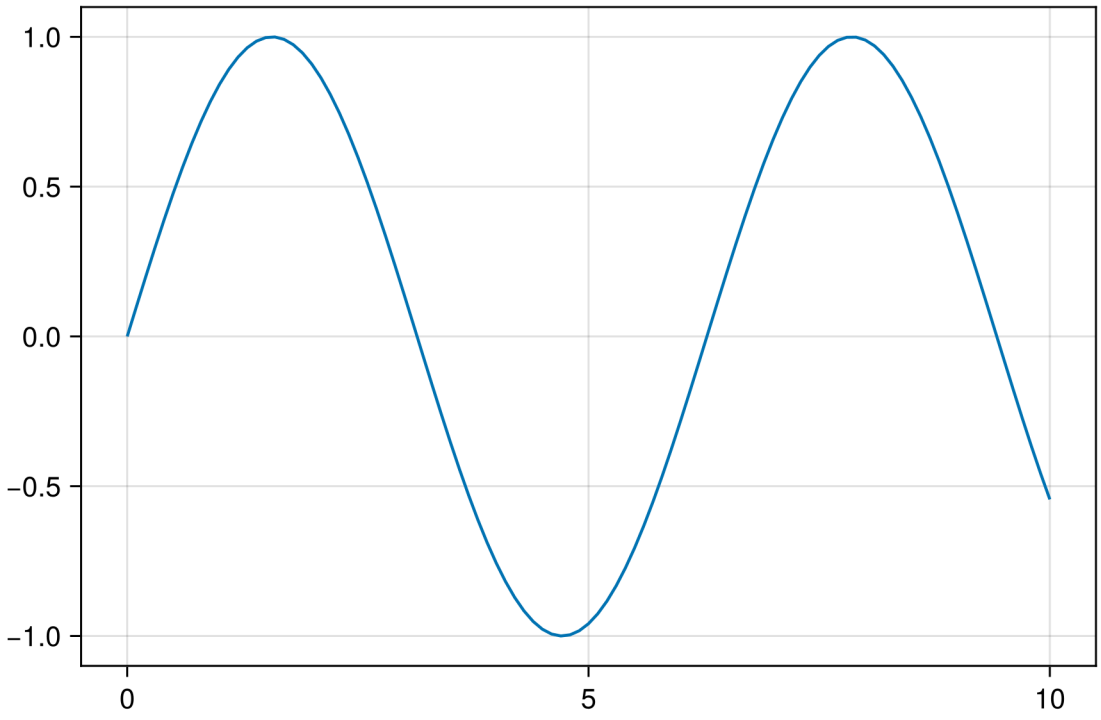
> **First load is slow, that is normal**
>
> The very first time you run `using CairoMakie` in a fresh Julia session, it takes a while (sometimes 20 to 40 seconds) because Julia compiles the package. Subsequent calls in the same session are instant. This is Julia's "time to first plot" (TTFP). It has improved enormously over recent versions and continues to get faster.

```
using CairoMakie
CairoMakie.activate!(type = "png")
```

## 3.3 Your first plot

Let's start with the simplest possible plot, a line:

```
x = 0:0.1:10
y = sin.(x)

lines(x, y)
```

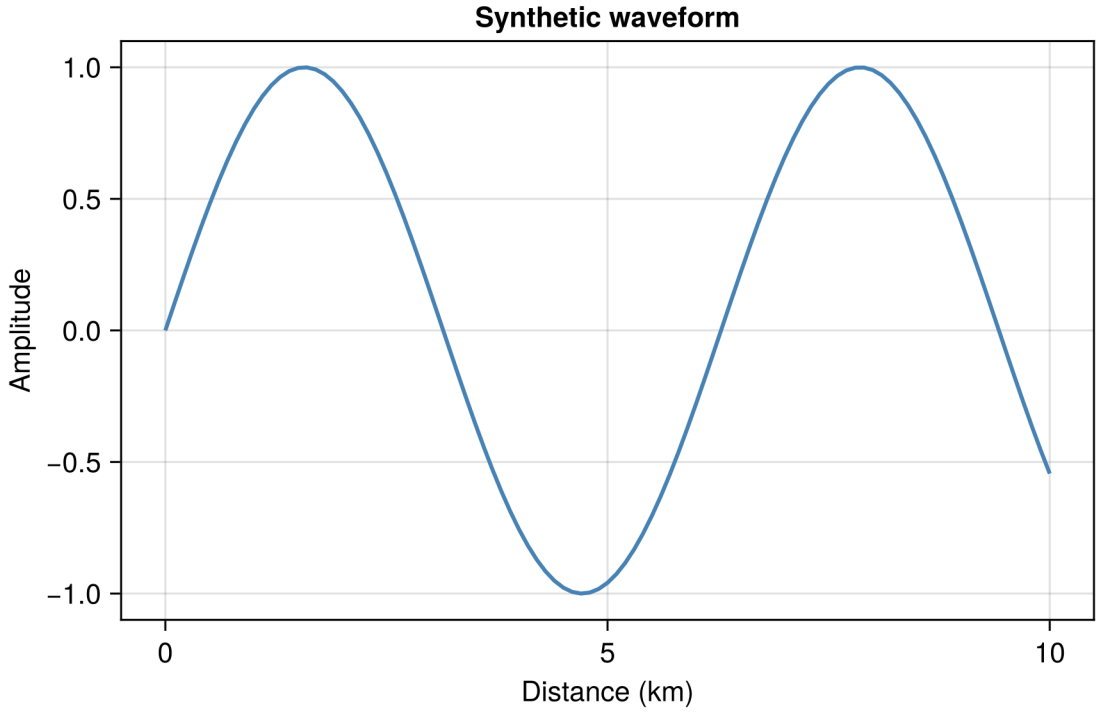



That is it. `lines(x, y)` creates a figure, adds an axis, and draws a line through the points. Makie returns a `FigureAxisPlot` object that renders automatically.

### 3.3.1 Adding labels and a title

A plot without labels is just a pretty picture. Always label your axes:

```
fig, ax, plt = lines(x, sin.(x),
    axis = (
        xlabel = "Distance (km)",
        ylabel = "Amplitude",
        title  = "Synthetic waveform"
    ),
    linewidth = 2,
    color = :steelblue
)
fig
```

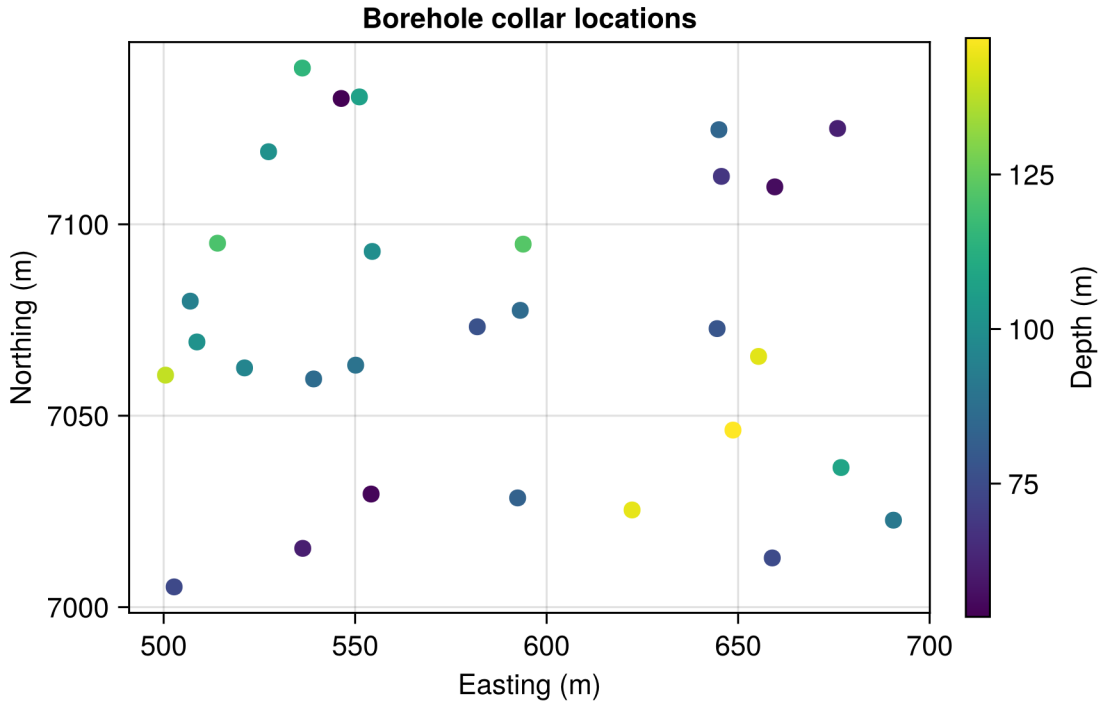



The `fig, ax, plt = ...` pattern gives you handles to the **figure**, the **axis**, and the **plot object**. You will use these handles a lot when building more complex figures.

## 3.4 Scatter plots

Scatter plots are the workhorse of geoscience: plotting measurement locations, geochemical concentrations, geophysical anomalies, or any point data.

```
# Synthetic borehole collar locations
n = 30
easting  = 500 .+ 200 .* rand(n)
northing = 7000 .+ 150 .* rand(n)
depth    = 50 .+ 100 .* rand(n)

fig, ax, plt = scatter(easting, northing,
    color = depth,
    colormap = :viridis,
    markersize = 12,
    axis = (
        xlabel = "Easting (m)",
        ylabel = "Northing (m)",
        title  = "Borehole collar locations",
        aspect = DataAspect()
    )
)
Colorbar(fig[1, 2], plt, label = "Depth (m)")
fig
```




A few things to notice:

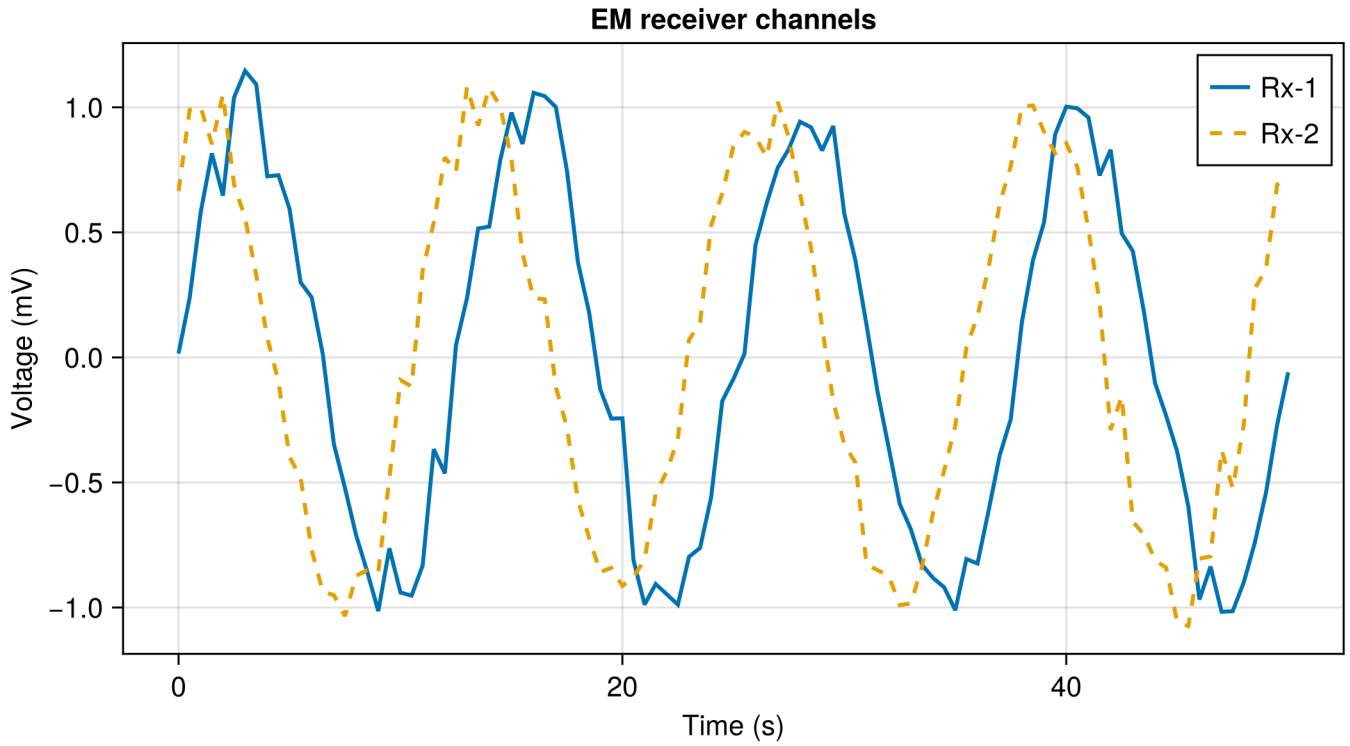
- **`color = depth`** maps a data vector to colour, just like you would colour-code sample values on a field map.

- **`colormap = :viridis`** picks a perceptually uniform colourmap. Other good choices for geoscience: `:turbo`, `:RdBu` (diverging, great for anomalies), `:terrain`, `:deep`.
- **`Colorbar(...)`** adds a colour legend. The position `fig[1, 2]` means "row 1, column 2 of the figure layout."
- **`aspect = DataAspect()`** keeps the x and y scales equal — essential when plotting spatial coordinates.

## 3.5 Line plots with multiple series

Geoscientists often compare several datasets on the same axes: different field campaigns, model predictions versus observations, or time series from multiple stations.

```
time = 0:0.5:50            # time in seconds
signal_1 = sin.(0.5 .* time) .+ 0.1 .* randn(length(time))
signal_2 = sin.(0.5 .* time .+ π/3) .+ 0.1 .* randn(length(time))

fig = Figure(size = (700, 400))
ax = Axis(fig[1, 1],
    xlabel = "Time (s)",
    ylabel = "Voltage (mV)",
    title  = "EM receiver channels"
)
lines!(ax, time, signal_1, label = "Rx-1", linewidth = 2)
lines!(ax, time, signal_2, label = "Rx-2", linewidth = 2, linestyle = :dash)
axislegend(ax, position = :rt)
fig
```

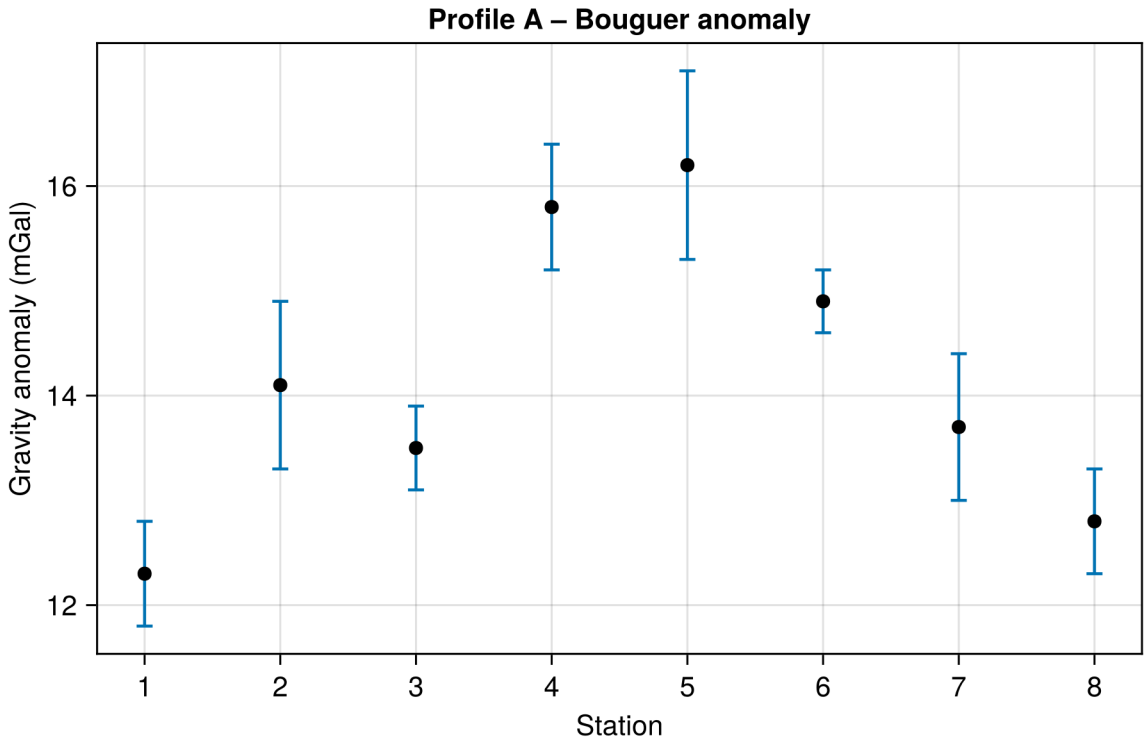



Notice the `!` in `lines!` and `axislegend!`-style calls. The exclamation mark means "add to an existing axis" rather than creating a new one. This is a consistent Makie convention.

## 3.6 Error bars

Field measurements always have uncertainty. Use `errorbars!` to show it:

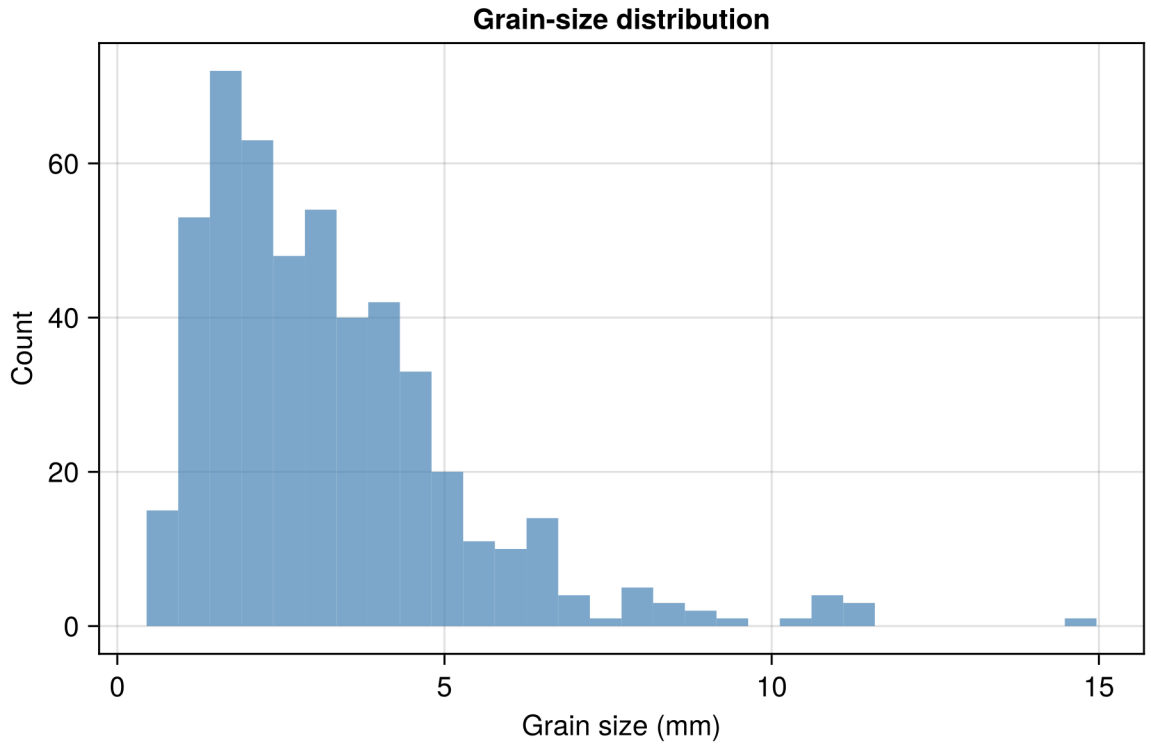
```
stations = 1:8
gravity  = [12.3, 14.1, 13.5, 15.8, 16.2, 14.9, 13.7, 12.8]
uncert   = [0.5,  0.8,  0.4,  0.6,  0.9,  0.3,  0.7,  0.5]

fig = Figure(size = (600, 400))
ax = Axis(fig[1, 1],
    xlabel = "Station",
    ylabel = "Gravity anomaly (mGal)",
    title  = "Profile A – Bouguer anomaly",
    xticks = 1:8
)
errorbars!(ax, stations, gravity, uncert, whiskerwidth = 8)
scatter!(ax, stations, gravity, markersize = 10, color = :black)
fig
```




## 3.7 Bar charts and histograms

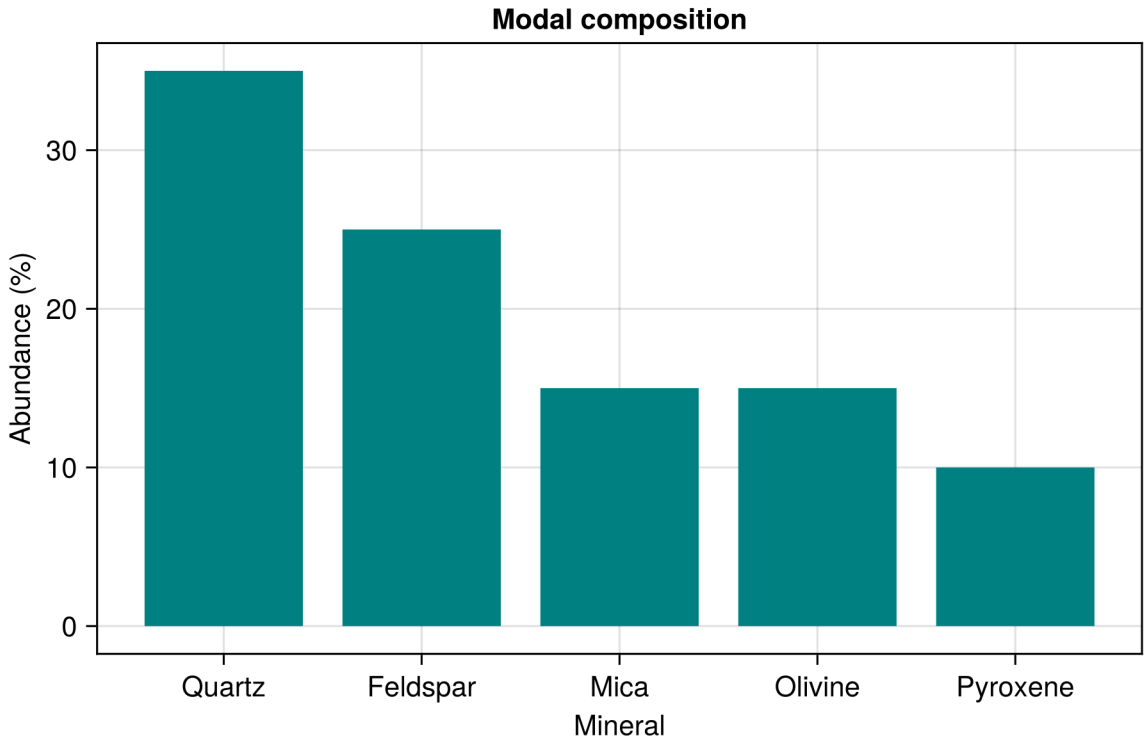
### 3.7.1 Histogram of sample data

```
# Simulated grain sizes (log-normal distribution)
grain_sizes = exp.(randn(500) .* 0.6 .+ 1.0)

fig = Figure(size = (600, 400))
ax = Axis(fig[1, 1],
    xlabel = "Grain size (mm)",
    ylabel = "Count",
    title  = "Grain-size distribution"
)
hist!(ax, grain_sizes, bins = 30, color = (:steelblue, 0.7))
fig
```




### 3.7.2 Bar chart

```
minerals  = ["Quartz", "Feldspar", "Mica", "Olivine", "Pyroxene"]
abundance = [35, 25, 15, 15, 10]

fig = Figure(size = (600, 400))
ax = Axis(fig[1, 1],
    xlabel = "Mineral",
    ylabel = "Abundance (%)",
    title  = "Modal composition",
    xticks = (1:5, minerals)
)
barplot!(ax, 1:5, abundance, color = :teal)
fig
```



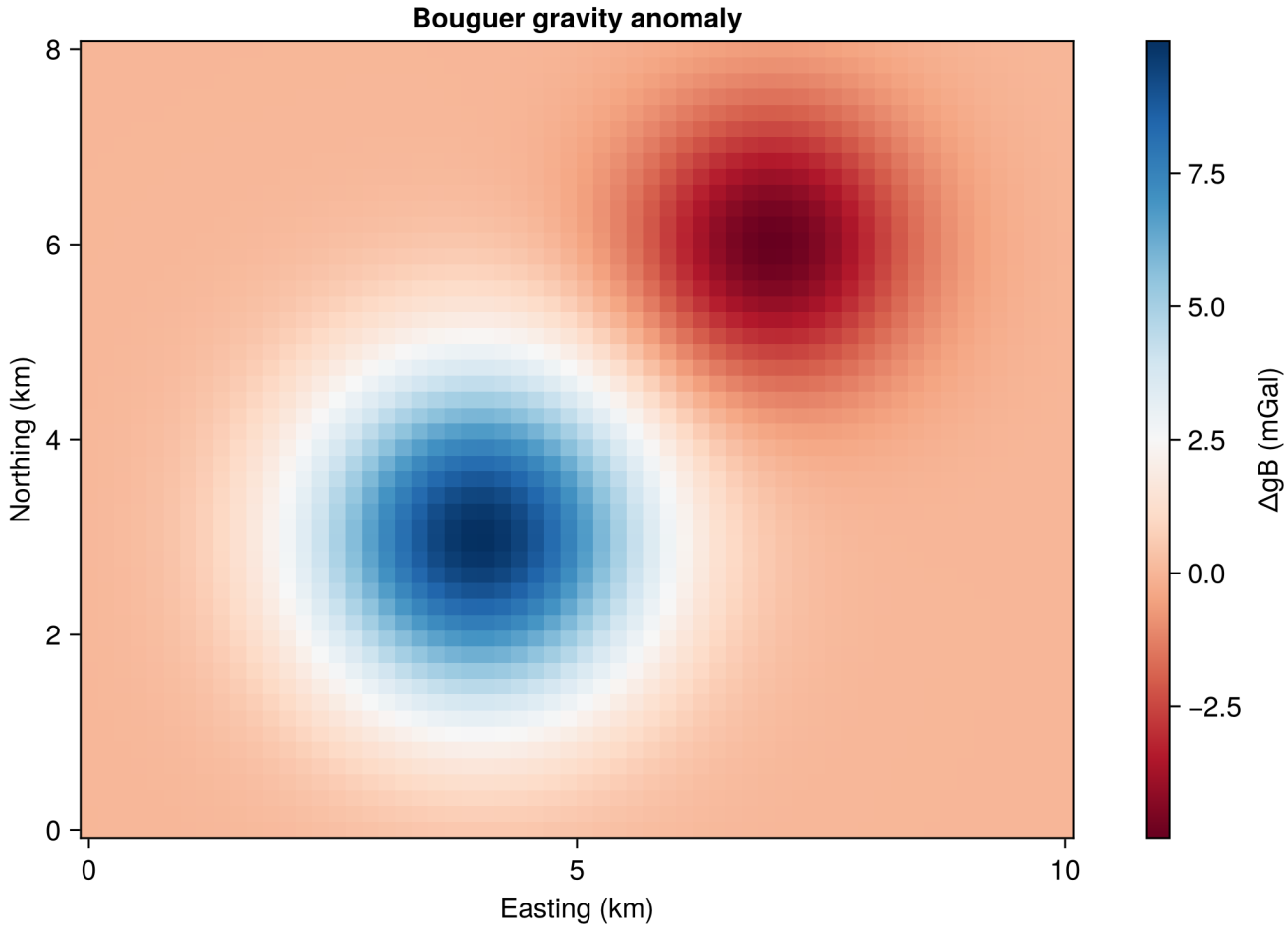

## 3.8 Heatmaps and contour plots

Gridded data is everywhere in geoscience: gravity grids, magnetic surveys, temperature fields, elevation models. Makie handles these with `heatmap` and `contour`/`contourf`.

### 3.8.1 Heatmap

```
# Synthetic gravity anomaly on a 2-D grid
x_grid = range(0, 10, length = 60)
y_grid = range(0, 8,  length = 50)
gz = [10 * exp(-((xi - 4)^2 + (yi - 3)^2) / 3) -
       5 * exp(-((xi - 7)^2 + (yi - 6)^2) / 2)
      for xi in x_grid, yi in y_grid]

fig = Figure(size = (700, 500))
ax = Axis(fig[1, 1],
    xlabel = "Easting (km)",
    ylabel = "Northing (km)",
    title  = "Bouguer gravity anomaly",
    aspect = DataAspect()
)
```

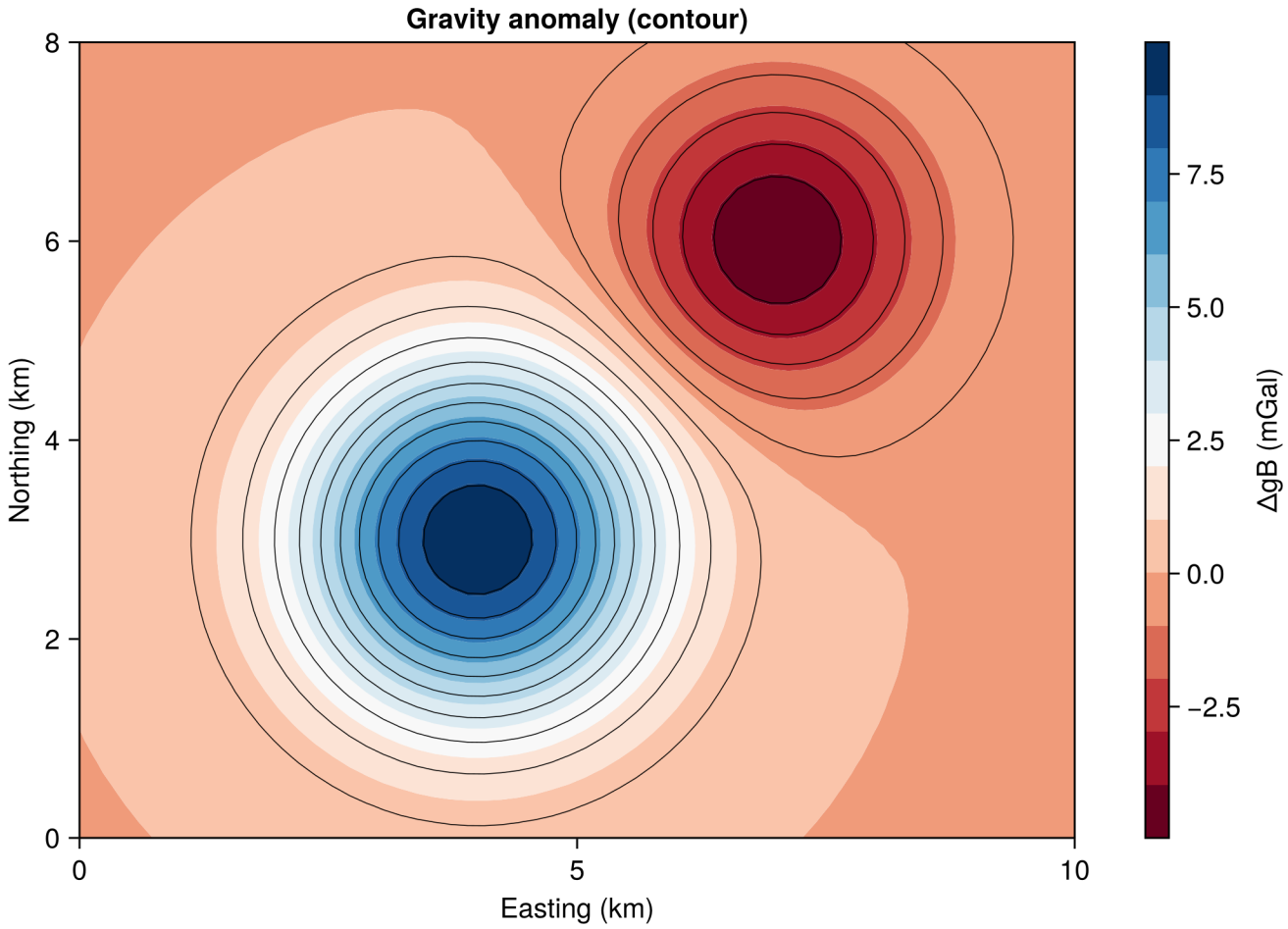
```
hm = heatmap!(ax, x_grid, y_grid, gz, colormap = :RdBu)
Colorbar(fig[1, 2], hm, label = "ΔgB (mGal)")
fig
```




### 3.8.2 Filled contour plot

```
fig = Figure(size = (700, 500))
ax = Axis(fig[1, 1],
    xlabel = "Easting (km)",
    ylabel = "Northing (km)",
    title  = "Gravity anomaly (contour)",
    aspect = DataAspect()
)
co = contourf!(ax, x_grid, y_grid, gz, levels = 15, colormap = :RdBu)
contour!(ax, x_grid, y_grid, gz, levels = 15, color = :black, linewidth = 0.5)
Colorbar(fig[1, 2], co, label = "ΔgB (mGal)")
```

```
fig
```



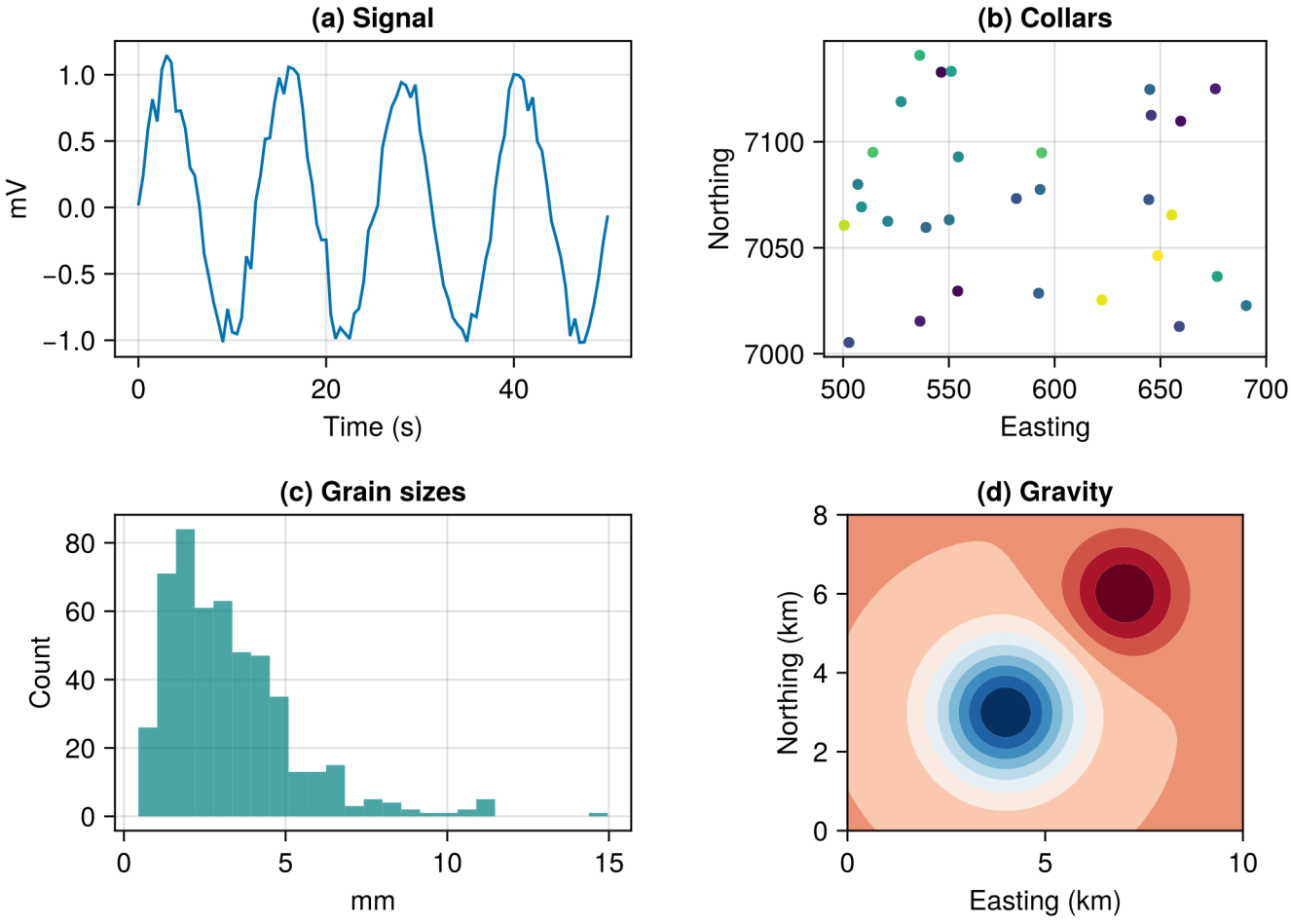

Overlaying `contour!` lines on a `contourf!` fill is a classic geophysics style. The thin black lines make it easier to read precise values.

## 3.9 Multi-panel figures (subplots)

Scientific papers often require several panels in one figure. Makie uses a **grid layout** system: `fig[row, col]` places content anywhere on the grid.

```
fig = Figure(size = (700, 500))

# Panel a — line plot
ax1 = Axis(fig[1, 1], title = "(a) Signal", xlabel = "Time (s)", ylabel = "mV")
lines!(ax1, time, signal_1)
```

```
# Panel b — scatter
ax2 = Axis(fig[1, 2], title = "(b) Collars",
    xlabel = "Easting", ylabel = "Northing", aspect = DataAspect())
scatter!(ax2, easting, northing, color = depth, colormap = :viridis, markersize = 8)

# Panel c — histogram
ax3 = Axis(fig[2, 1], title = "(c) Grain sizes", xlabel = "mm", ylabel = "Count")
hist!(ax3, grain_sizes, bins = 25, color = (:teal, 0.7))

# Panel d — contour
ax4 = Axis(fig[2, 2], title = "(d) Gravity", xlabel = "Easting (km)",
    ylabel = "Northing (km)", aspect = DataAspect())
contourf!(ax4, x_grid, y_grid, gz, levels = 12, colormap = :RdBu)

fig
```



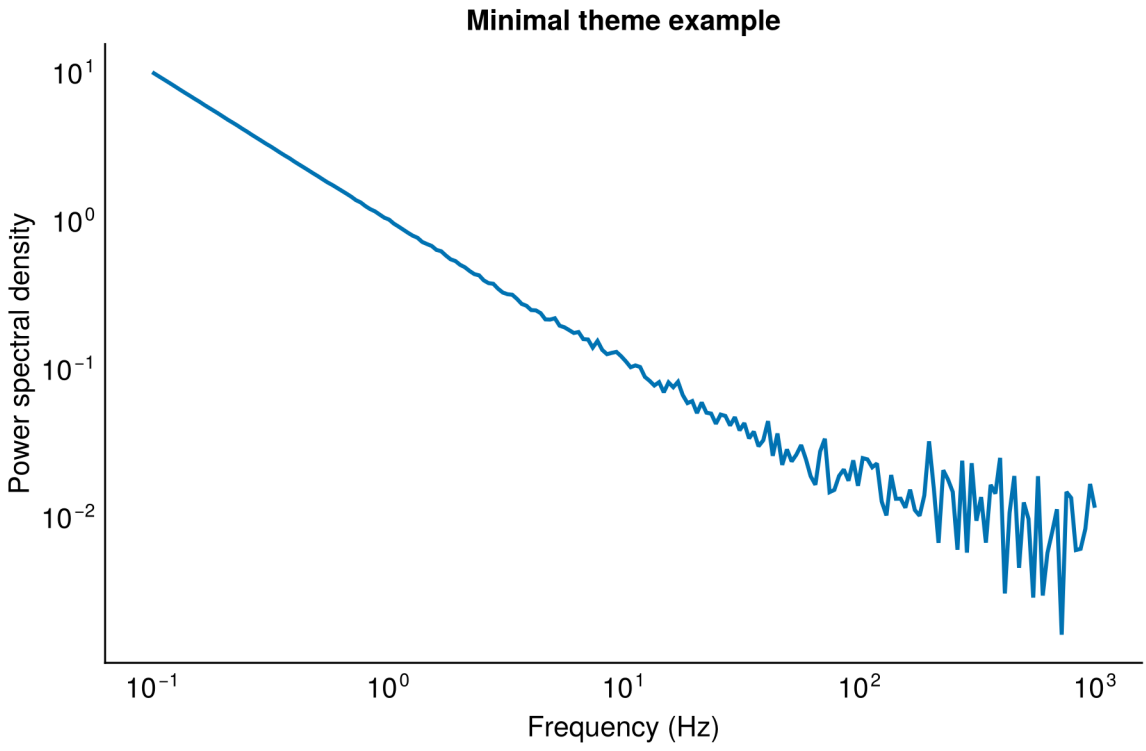

You can span rows or columns for insets and wide panels — see the Makie layout tutorial for advanced arrangements.

## 3.10 Customising appearance

### 3.10.1 Fonts and themes

Makie ships with several built-in themes. You can also override individual settings:

```
with_theme(theme_minimal()) do
    fig = Figure(size = (600, 400))
    ax = Axis(fig[1, 1],
        xlabel = "Frequency (Hz)",
        ylabel = "Power spectral density",
        title  = "Minimal theme example",
        xscale = log10,
        yscale = log10
    )
    freqs = 10 .^ range(-1, 3, length = 200)
    psd   = 1 ./ freqs .+ 0.01 .* abs.(randn(200))
    lines!(ax, freqs, psd, linewidth = 2)
    fig
end
```



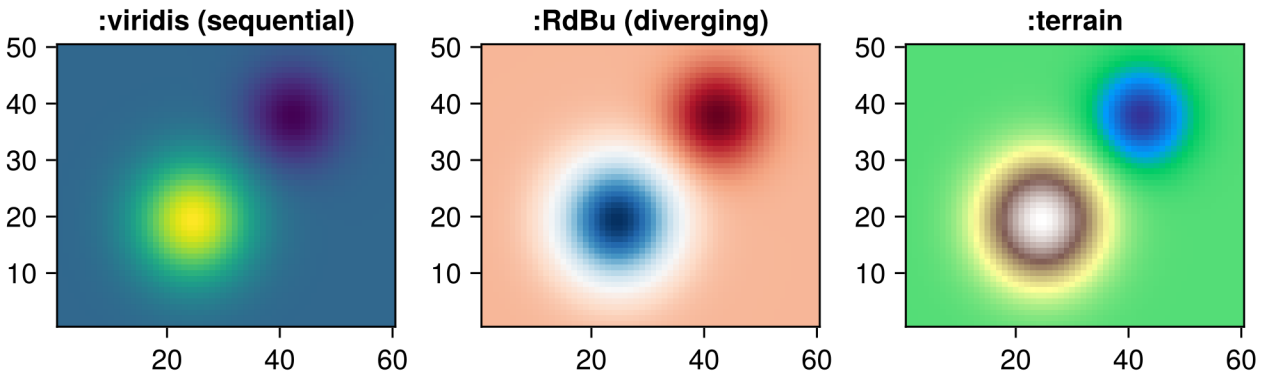

Other themes to try: `theme_dark()`, `theme_light()`, `theme_black()`, `theme_ggplot2()`.

### 3.10.2 Colourmaps

Choosing the right colourmap matters, especially in geoscience where a bad colourmap can hide features or mislead the reader. Some guidelines:

| Data type | Good choices | Avoid |
|---|---|---|
| Sequential (elevation, depth, concentration) | `:viridis`, `:cividis`, `:deep`, `:thermal` | `:jet` (perceptual jumps) |
| Diverging (anomalies centred on zero) | `:RdBu`, `:BrBG`, `:PiYG` | Any sequential map |
| Categorical (lithology, fault type) | `:tab10`, `:Set1` | Continuous maps |
| Terrain / bathymetry | `:terrain`, `:topo` | `:hot`, `:gray` |

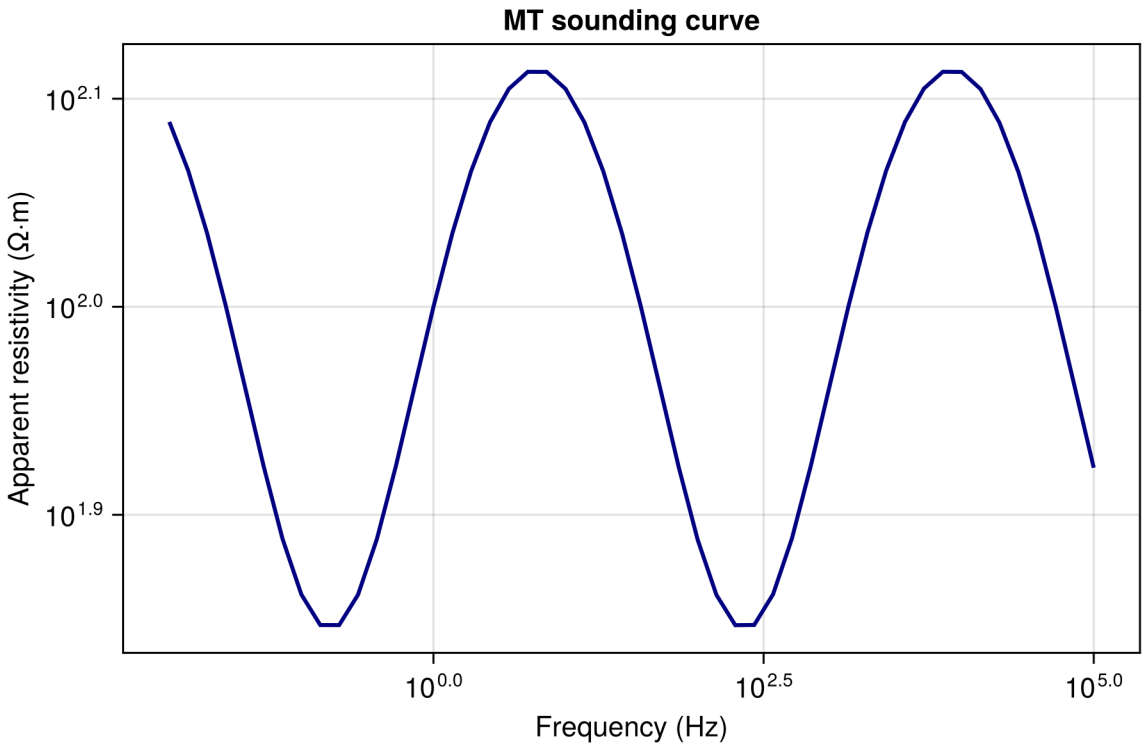
```
fig = Figure(size = (650, 280))

ax1 = Axis(fig[1, 1], title = ":viridis (sequential)", aspect = DataAspect())
heatmap!(ax1, gz, colormap = :viridis)

ax2 = Axis(fig[1, 2], title = ":RdBu (diverging)", aspect = DataAspect())
heatmap!(ax2, gz, colormap = :RdBu)

ax3 = Axis(fig[1, 3], title = ":terrain", aspect = DataAspect())
heatmap!(ax3, gz, colormap = :terrain)

fig
```



## 3.11 Log-scale axes

Geophysical data often spans many orders of magnitude: resistivity, frequency, spectral power. Use `xscale = log10` or `yscale = log10`:

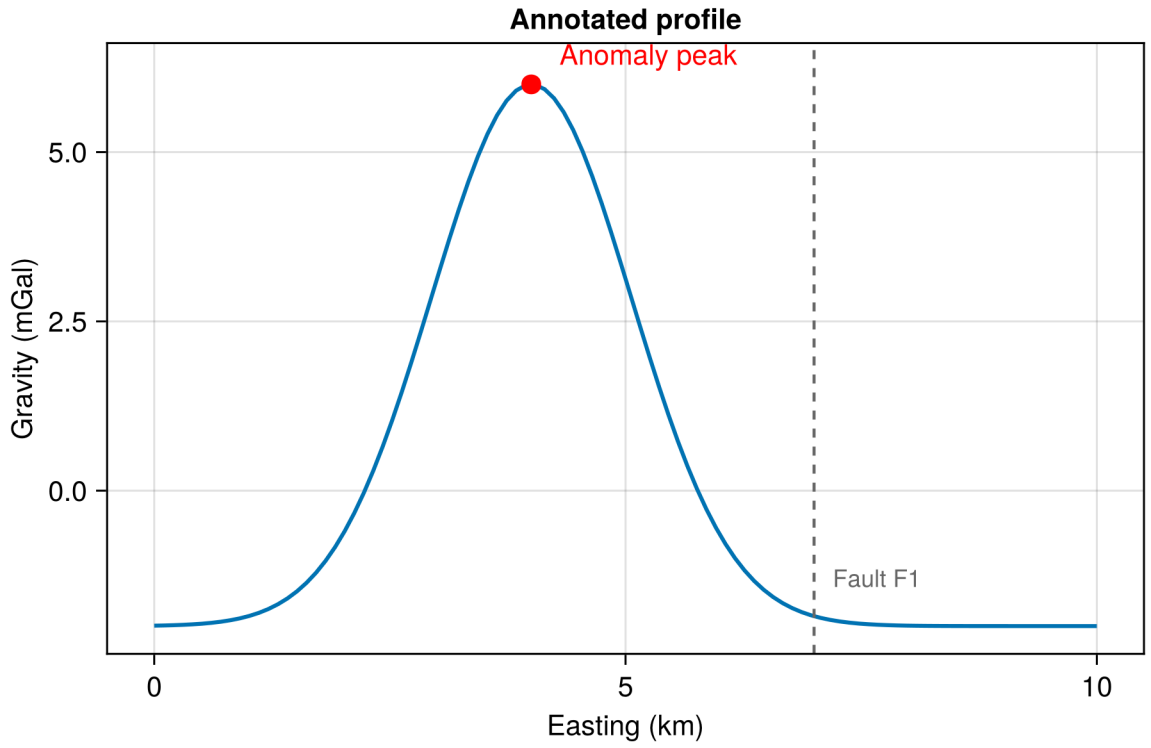
```
freq = 10 .^ range(-2, 5, length = 50)
apparent_res = 100 .* (1 .+ 0.3 .* sin.(log10.(freq) .* 2))

fig = Figure(size = (600, 400))
ax = Axis(fig[1, 1],
    xlabel = "Frequency (Hz)",
    ylabel = "Apparent resistivity (Ω·m)",
    title  = "MT sounding curve",
    xscale = log10,
    yscale = log10
)
lines!(ax, freq, apparent_res, linewidth = 2, color = :navy)
fig
```



## 3.12 Annotations and markers

Labelling key features on a plot (a fault, an anomaly peak, a sample location) is something you will do often:

```
fig = Figure(size = (600, 400))
ax = Axis(fig[1, 1],
    xlabel = "Easting (km)", ylabel = "Gravity (mGal)",
    title  = "Annotated profile"
)
profile_x = range(0, 10, length = 100)
profile_g = 8 .* exp.(-((profile_x .- 4) ./ 1.5).^2) .- 2
lines!(ax, profile_x, profile_g, linewidth = 2)

# Mark the peak
scatter!(ax, [4.0], [6.0], markersize = 14, color = :red)
text!(ax, 4.3, 6.2, text = "Anomaly peak", fontsize = 14, color = :red)

# Mark a fault
vlines!(ax, [7.0], color = :grey40, linestyle = :dash, linewidth = 1.5)
text!(ax, 7.2, -1.5, text = "Fault F1", fontsize = 12, color = :grey40)

fig
```



## 3.13 Saving figures

Use `save` to export your figure. The file extension determines the format:

```
save("gravity_profile.png", fig)              # raster (300 dpi by default)
save("gravity_profile.pdf", fig)              # vector, best for papers
save("gravity_profile.svg", fig)              # vector, good for web
save("gravity_profile.png", fig, px_per_unit = 3)  # high-res raster (900 dpi)
```

PDF for papers, PNG for presentations

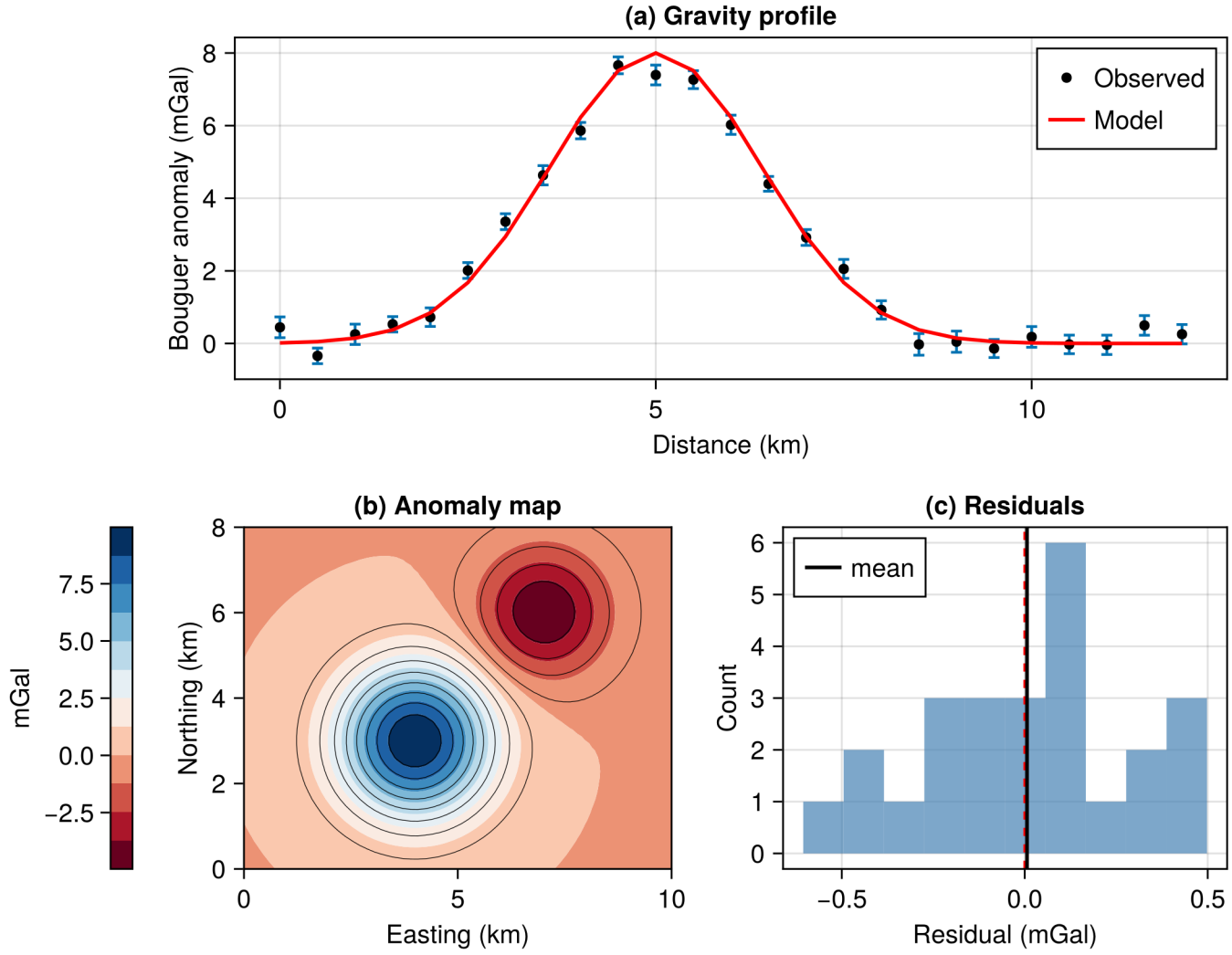
For journal submissions, **PDF** or **SVG** gives you lossless vector graphics that look sharp at any zoom. For slides and posters, **PNG at high resolution** (`px_per_unit = 3` or higher) is more portable.

## 3.14 Putting it together: a geoscience figure

Let's combine several techniques into a single multi-panel figure typical of a geophysical report:

```
using Statistics

fig = Figure(size = (700, 550))

# --- Panel a: profile with error bars ---
ax1 = Axis(fig[1, 1:2], title = "(a) Gravity profile",
    xlabel = "Distance (km)", ylabel = "Bouguer anomaly (mGal)")
dist   = range(0, 12, length = 25)
obs    = 8 .* exp.(-((dist .- 5) ./ 2).^2) .+ 0.3 .* randn(25)
uncert = 0.2 .+ 0.1 .* rand(25)
errorbars!(ax1, collect(dist), obs, uncert, whiskerwidth = 6)
scatter!(ax1, collect(dist), obs, markersize = 8, color = :black, label = "Observed")
model_g = 8 .* exp.(-((dist .- 5) ./ 2).^2)
lines!(ax1, collect(dist), model_g, linewidth = 2, color = :red, label = "Model")
axislegend(ax1, position = :rt)

# --- Panel b: contour map ---
ax2 = Axis(fig[2, 1], title = "(b) Anomaly map",
    xlabel = "Easting (km)", ylabel = "Northing (km)",
    aspect = DataAspect())
co = contourf!(ax2, x_grid, y_grid, gz, levels = 12, colormap = :RdBu)
contour!(ax2, x_grid, y_grid, gz, levels = 12, color = :black, linewidth = 0.4)
Colorbar(fig[2, 0], co, label = "mGal", flipaxis = false)

# --- Panel c: histogram of residuals ---
residuals = obs .- model_g
```

```
ax3 = Axis(fig[2, 2], title = "(c) Residuals",
    xlabel = "Residual (mGal)", ylabel = "Count")
hist!(ax3, residuals, bins = 10, color = (:steelblue, 0.7))
vlines!(ax3, [0.0], color = :red, linestyle = :dash)
vlines!(ax3, [mean(residuals)], color = :black, linewidth = 2, label = "mean")
axislegend(ax3, position = :lt)

fig
```




## 3.15 Geographic maps with GeoMakie

So far every plot has used plain numeric axes — "Easting (km)" and "Northing (km)." But geoscientists often need to plot data on a **map** with coastlines, country borders, and a proper geographic projection. That is

what **GeoMakie.jl** gives you.

GeoMakie provides a `GeoAxis`, a drop-in replacement for Makie's `Axis` that understands coordinate reference systems (CRS) and projections. You feed it longitude/latitude data and tell it which projection you want; it handles the transformation and draws graticules automatically.

### 3.15.1 Installing GeoMakie

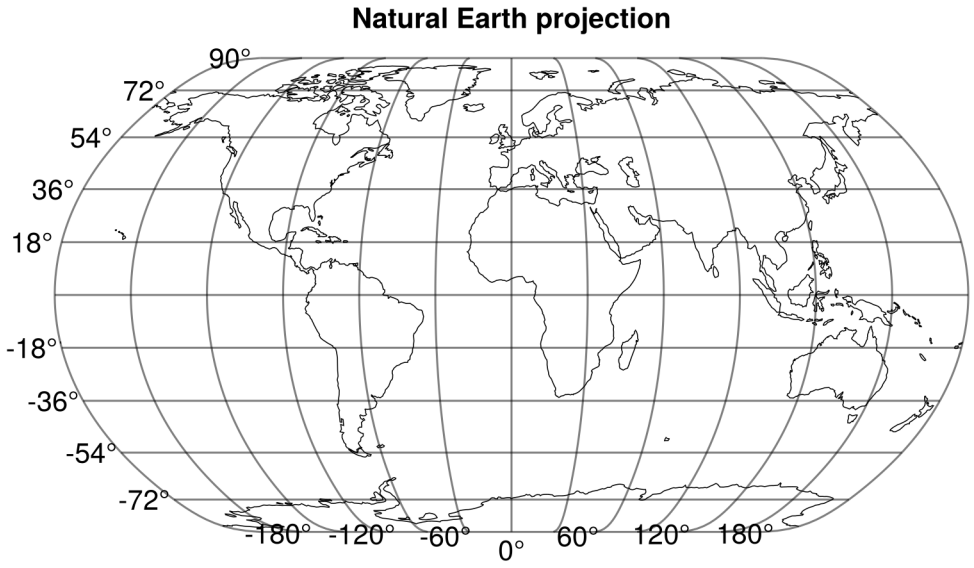
```
pkg> add GeoMakie
```

Or: `using Pkg; Pkg.add("GeoMakie")`. GeoMakie pulls in Proj.jl (for map projections) and NaturalEarth (for coastlines and country boundaries).

```
using GeoMakie
```

### 3.15.2 A world map in three lines

```
fig = Figure()
ga = GeoAxis(fig[1, 1], dest = "+proj=natearth", title = "Natural Earth projection")
lines!(ga, GeoMakie.coastlines(), color = :black, linewidth = 0.5)
fig
```



`GeoAxis` accepts any PROJ-string through the `dest` keyword. The default source CRS is WGS84 (longitude/latitude). Try replacing `"+proj=natearth"` with `"+proj=ortho"` for an orthographic globe view, or `"+proj=moll"` for a Mollweide equal-area projection.

### 3.15.3 Overlaying polygons and raster data

GeoMakie integrates with **GeoInterface.jl**, so you can plot any GeoInterface-compatible geometry: shapefiles, GeoJSON, geological unit polygons, fault traces, concession boundaries:

```
using GeoMakie, GeoJSON

# Load a GeoJSON of geological units (hypothetical file)
geol = GeoJSON.read("geology_units.geojson")

fig = Figure()
ga = GeoAxis(fig[1, 1], dest = "+proj=utm +zone=18 +south")
poly!(ga, geol, color = :lightgrey, strokecolor = :black, strokewidth = 0.5)
fig
```

Common projections for geoscience

| Region / Use case | PROJ string | Notes |
|---|---|---|
| Global equal-area | `+proj=moll` | Mollweide, good for thematic world maps |
| Global for visuals | `+proj=natearth` | Natural Earth, pleasant for presentations |
| Globe view | `+proj=ortho +lon_0=0 +lat_0=30` | Orthographic, looks like a photo from space |
| Local field survey | `+proj=utm +zone=35 +datum=WGS84` | Metric grid, precise local coordinates |
| Continental | `+proj=lcc +lat_1=30 +lat_2=60 +lon_0=10` | Lambert Conformal Conic, preserves shape |
| Polar | `+proj=stere +lat_0=90 +lon_0=0` | Stereographic, Arctic/Antarctic maps |

> You can browse all available projections at https://proj.org/operations/projections/.

### 3.15.4 Learn more about GeoMakie

- **Documentation & gallery**: https://geo.makie.org/
- **Source code**: https://github.com/MakieOrg/GeoMakie.jl

## 3.16 Where to learn more

- **Makie documentation**: https://docs.makie.org/ with tutorials, gallery, and the full API reference.
- **Beautiful Makie**: https://beautiful.makie.org/, a gallery of polished example figures you can copy and adapt.
- **Makie colour maps**: the `ColorSchemes.jl` package gives you access to hundreds of colour maps. Browse them at https://juliagraphics.github.io/ColorSchemes.jl/stable/.

# Part II

# Neural Networks

# 4 Neural Networks

At this point you know how to handle data files, manipulate tables, and create plots.

In this chapter we will build **neural networks** that learn patterns directly from data. We start with a hands-on example (generate data, train, evaluate) before unpacking the theory. The framework is **Lux.jl**, a modern Julia library that keeps model parameters and state explicit.

## 4.1 Sauna Satisfaction Predictor

Let's skip the theory and train a neural network end to end. In an alternate universe, the Geological Survey of Finland maintains sauna visitor logs. For each visit, three inputs are recorded:

- **Sauna temperature** (°C)
- **Outside temperature** (°C)

- **Minutes since last coffee**

After the session, the visitor assigns a **Satisfaction Score** between 0 and 1. We will train a small neural network to predict this score from the three inputs.

```
using Lux, Random, Optimisers, Zygote, Statistics, Printf
```

First, read the data and normalize inputs to zero mean and unit variance:

```
lines = readlines("sauna_log.txt")
data = reduce(hcat, [parse.(Float32, split(l, ",")) for l in lines[2:end]])

X_raw = data[1:3, :]
Y     = data[4:4, :]

# Normalize inputs
mu  = mean(X_raw, dims = 2)
sig = std(X_raw, dims = 2)
X   = (X_raw .- mu) ./ sig

# Split into train and test
rng = Xoshiro(123)
n_train = Int(round(0.8 * size(X, 2)))
idx = randperm(rng, size(X, 2))
```

```
X_train = X[:, idx[1:n_train]]
Y_train = Y[:, idx[1:n_train]]
X_test  = X[:, idx[n_train+1:end]]
Y_test  = Y[:, idx[n_train+1:end]]

@printf "Train: %d  Test: %d\n" size(X_train, 2) size(X_test, 2)
```

```
Train: 160  Test: 40
```

Build a simple neural network with one hidden layer (8 neurons with tanh activation):

```
model = Chain(
    Dense(3 => 8, tanh),
    Dense(8 => 1)
)

# Initialize parameters and state
ps, st = Lux.setup(rng, model)

# Define loss function (mean squared error)
function mse_loss(model, ps, st, data)
    x, y_true = data
    y_pred, st_new = model(x, ps, st)
    loss = mean((y_pred .- y_true) .^ 2)
    return loss, st_new, ()
end
```

```
mse_loss (generic function with 1 method)
```

Train the network using the Adam optimizer:

```
opt = Adam(0.01f0)
tstate = Training.TrainState(model, ps, st, opt)

for epoch in 1:500
    _, loss, _, tstate = Training.single_train_step!(
        AutoZygote(), mse_loss, (X_train, Y_train), tstate
    )
    if epoch == 1 || epoch % 100 == 0
        @printf "Epoch %4d  MSE = %.6f\n" epoch loss
    end
end

# Evaluate on test data
```

```
Y_pred_test, _ = model(X_test, tstate.parameters, tstate.states)
test_mse = mean((Y_pred_test .- Y_test) .^ 2)
test_mae = mean(abs.(Y_pred_test .- Y_test))

@printf "Test MSE: %.6f  Test MAE: %.4f\n" test_mse test_mae
```

```
Epoch    1  MSE = 0.298768
Epoch  100  MSE = 0.021929
Epoch  200  MSE = 0.008098
Epoch  300  MSE = 0.006308
Epoch  400  MSE = 0.005393
Epoch  500  MSE = 0.005107
Test MSE: 0.006323  Test MAE: 0.0615
```

Finally, make predictions for specific scenarios:

```
scenarios = [
    (sauna=80, outside=-20, coffee=10, label="Perfect: 80°C, -20°C, fresh coffee"),
    (sauna=65, outside=5,   coffee=100, label="Poor: lukewarm, mild, stale coffee"),
    (sauna=95, outside=-25, coffee=30, label="Extreme: 95°C, deep winter"),
]

function predict_score(s, model, ps, st, mu, sig)
    x = Float32.([(s.sauna - mu[1]) / sig[1],
                  (s.outside - mu[2]) / sig[2],
                  (s.coffee - mu[3]) / sig[3]])
    pred, _ = model(reshape(x, 3, 1), ps, st)
    return pred[1]
end

for s in scenarios
    sc = predict_score(s, model, tstate.parameters, tstate.states, mu, sig)
    @printf "%-35s %.2f\n" s.label sc
end
```

```
Perfect: 80°C, -20°C, fresh coffee  1.04
Poor: lukewarm, mild, stale coffee  0.07
Extreme: 95°C, deep winter          0.41
```

## 4.2 What is a neural network?

A **neuron** multiplies inputs by weights, adds bias, and applies non-linear activation:

$$z = \sigma\left(\mathbf{w}^\top \mathbf{x} + b\right)$$

Stack neurons to get a **layer**; chain layers to get a **network**. Learning happens through four repeated steps: forward pass, compute loss, backward pass (gradients), and parameter update.

> **Work in progress**
>
> The next chapters will cover scientific machine learning in detail, combining domain knowledge with data-driven approaches for geoscientific problems.

# Part III

# Scientific Machine Learning

# 5 Inverse Modelling

> **Work in progress**
>
> This chapter is under development.

# Part IV

# Applications

# 6 Applications

> **Work in progress**
>
> This chapter is under development.

# References

Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A fresh approach to numerical computing. *SIAM Review*, *59*(1), 65–98. https://doi.org/10.1137/141000671